Software Engineering Department

Braude College of Engineering

Final Project – Phase A (61998)

OmniSight

Distributed Semantic Video Intelligence
& Predictive Reasoning Platform

CODE: 26-2-D-17

By:
Ahmad Tawil
Cyrine Fahoum

Advisor:

Dr. Reuven Cohen

Link to GitHub:
https://github.com/OmniSight-team/omni-sight

Table of Contents

Abstract

Video is now one of the world's most abundant data sources: surveillance networks, sports broadcasts, and traffic cameras record continuously. But this footage is far easier to collect than to use. Finding a single moment in it — when someone carried a suspicious object into the building — can take hours of manual review, and spotting an event as it unfolds — a car drifting out of its lane — is harder still. As archives grow faster than anyone can watch them, incidents are missed, insights arrive late, and investigators rebuild timelines by hand. The core problem is that searching and analyzing long videos by what actually happens in it is still too slow and too fragmented to support decisions while they still matter.

OmniSight is a video-intelligence platform that addresses this problem in a single end-to-end pipeline. It ingests live and recorded video, indexes it by meaning, and lets users search in everyday language. Beyond retrieving what has already happened, a Predictive Forecasting Module (PFM) recognizes the early signs of an unfolding event and raises a warning before it happens; a layered verification cascade filters out false alarms; and a retrieval-augmented module assembles multi-camera footage into evidence-grounded, fully cited incident reports. All capabilities are available through a web dashboard and to external AI agents via the Model Context Protocol (MCP). The project's main empirical contribution is the PFM — a single temporal model shared across surveillance, sports, and traffic, whose ability to transfer across these domains is measured directly rather than assumed.

1. Introduction

Video has become one of the most abundant data sources in the modern world, with surveillance networks, sports broadcasts, and traffic cameras recording continuously. Yet finding a specific moment in this footage — a package left at a doorway or the seconds before a collision — often takes hours of manual review. As archives grow faster than humans can watch them, incidents are missed, insights arrive late, and investigators reconstruct timelines by hand. Video is recorded everywhere, yet practically unusable.

The problem this project addresses is that long-form video cannot be indexed, searched, and analyzed quickly enough to support timely operational decisions. OmniSight is a distributed platform for semantic video intelligence that tackles this in a single end-to-end pipeline. It ingests live streams and uploaded video; indexes content by meaning rather than by file or timestamp; and lets users search using everyday language. Beyond retrieving what has already happened, a Predictive Forecasting Module recognizes early signs of an unfolding event and raises a warning before it completes. Each warning passes through a short verification cascade that filters obvious false alarms. When an incident occurs, the system assembles footage from one or more cameras into a structured summary that can be read in minutes. These capabilities are offered to people through a web dashboard and to other AI agents through the Model Context Protocol (MCP).

The project pursues five goals: (1) natural-language semantic search over surveillance, sports, and traffic archives, (2) forecasting of event type, time-to-event, and confidence with measurable early-warning lead time across all three domains using a shared temporal model, (3) false-alarm reduction through a layered verification cascade, (4) evidence-grounded multi-camera incident summaries, and (5) access to all core capabilities for external AI agents through a standard protocol.

The remainder of this document is organized as follows: Chapter 2 reviews the related literature; Chapter 3 presents the engineering process; and Chapter 4 defines the evaluation metrics and test plan.

2. Related Work

Modern video understanding builds on several research areas that were once studied mostly in isolation. This section reviews the five most relevant to this project: searching video with natural language, understanding motion over time, predicting what will happen next, detecting and explaining unusual events, and reconstructing past events from stored evidence.

2.1 Vision–Language Models for Open-Vocabulary Search

Traditional video analysis often assumes a fixed list of categories and can only detect those. This "closed set" setup is not flexible enough for searching, where a user might ask for almost anything. Open-vocabulary search (search over any words a user types, not just a fixed label list) removes this restriction and supports queries like "a person leaving a backpack on a bench".

The standard approach is to train a dual encoder (one network processes the image, another processes the text, both are trained so that matching image-text pairs produce similar outputs). This reduces search by comparing the text embedding with the embeddings of frames or clips. CLIP (Radford et al., 2021) set this pattern by training on about 400 million image-text pairs from the web, thereby achieving strong zero-shot performance (recognizing categories it never saw during training) across many domains. SigLIP 2 (Tschannen et al., 2025) improves on this idea by scoring each image-text pair independently rather than performing batch-wide comparisons, thereby stabilizing large-batch training and leading to better image-text alignment (see Appendix A.1 for architectural details).

Two main limitations remain. **First,** these models usually encode single frames in isolation, so they ignore that actions like "opening a door" and "closing a door" can look almost identical in a single frame. **Second**, training captions often mention only the most obvious objects, so the models are weaker at fine details, spatial relationships, and counting. These gaps motivate the addition of temporal modeling in Section 2.2 and more explicit reasoning in Section 2.4.

2.2 Temporal Video Understanding

Frame-level encoders treat a video as a bag of still images. In contrast, temporal video understanding focuses on motion and event structure that only appear when frames are viewed over time. This area has evolved through three main generations:

- Two-stream CNNs (Convolutional Neural Networks — pattern detectors that slide small filters over an image) process RGB frames and optical flow (per-pixel motion between frames) in parallel.
- 3D convolutional networks extend these filters into time, so they convolve over both space and time.
- Video transformers split a clip into space-time patches and connect them with self-attention (each patch assigns weights to all other patches to decide which are most relevant).

Training objectives have also shifted. Instead of relying solely on manual action labels, many models use self-supervised objectives (in which the model learns from raw, unlabeled data by predicting hidden parts from visible ones), such as masked video modeling (hiding parts of the video and training the model to reconstruct them). This produces general-purpose video foundation models. InternVideo2 (Wang et al., 2024) is a widely cited example: it uses staged training with masked reconstruction, multimodal matching, and next-token prediction, and reports strong results on more than sixty benchmarks (Appendix A.2).

Two limitations stand out. **First**, most models are trained on clips of just a few seconds. Extending their effective memory to minutes or hours — which is essential for long recordings — is still an open challenge. **Second**, the usual objectives focus on understanding what already happened rather than predicting what will happen, so event anticipation (Section 2.3) is often treated as a separate task.

2.3 Event Anticipation and Temporal Transformers

Event anticipation goes beyond recognition by predicting what is about to happen from a partially observed video. This is useful in driver assistance, proactive surveillance, and sports analysis, where reacting before an event matters more than recognizing it afterward.

Most methods use the Transformer (Vaswani et al., 2017) (a neural network design that lets each element in a sequence attend to every other element through self-attention, which measures how strongly each element depends on the rest). Here, the Transformer is applied along the time axis, so each moment in the video is compared with all other moments in the observed sequence. Three common approaches appear:

- Encoder–decoder transformers read the observed clip and output labels describing likely next actions.
- Predictive auto-encoders try to generate the future frames themselves. They first learn what a normal continuation of a scene looks like. When the predicted frames differ strongly from the actual future frames, the reconstruction error (the difference between prediction and reality) signals that something unusual may be about to happen.
- Trajectory transformers focus on movement rather than raw pixels. They track each agent (for example, a person or vehicle) and then forecast its likely future path.

These methods have been evaluated in several domains. Cao et al. (2023) introduced the NWPU Campus dataset, a large benchmark for anomaly anticipation (predicting abnormal events before they happen). Felsen et al. (2017) predicted basketball players' movements from their motion trajectories, and Yao et al. (2022) extended unsupervised anticipation (forecasting from unlabelled video with no explicit targets) to driving using the DoTA dataset.

The main limitation is that most benchmarks are single-domain. Each system is trained and tested in a single setting (for example, only on campus scenes, sports courts, or roads). As a result, it remains unclear how well the learned representations (the internal features reused across inputs) transfer to visually different environments. Cross-domain event anticipation remains under-explored.

2.4 Anomaly Detection and Causal Reasoning

Video anomaly detection focuses on segments that deviate from normal behaviors. Because anomalies are rare and varied, models cannot rely on a complete list of examples. Instead, they use weaker forms of supervision:

- Weakly supervised learning: labels only indicate that an anomaly occurs somewhere in a video, without precise timing.
- Self-supervised learning: the model learns what "normal" looks like from unlabeled footage and flags deviations.

UCF-Crime (Sultani et al., 2018), which contains about 1,900 CCTV videos, is a standard weakly supervised benchmark. More recent work moves from just detecting anomalies to explaining them as well. VADER (Cheng et al., 2026) combines object–relation features (what objects are present and how they interact) with a vision-language model to describe why an event is abnormal, not just that it is. The generative model behind this reasoning, Qwen2.5-VL (Bai et al., 2025), is detailed in Appendix A.3.

A key limitation is that this reasoning is usually post hoc, meaning the explanation is generated only after an anomaly has already been detected. Integrating causal reasoning into the anticipation stage — so that explanations accompany predictions rather than follow detections — is still an open research problem.

2.5 Retrieval-Augmented Generation for Forensic Reconstruction

Reconstructing events after an incident has traditionally required humans to build a timeline from scattered, heterogeneous evidence manually. Retrieval-Augmented Generation (RAG; combining database retrieval with text generation by a language model; Lewis et al., 2020) automates part of this process. Instead of relying solely on what the model memorized during training, RAG first retrieves relevant evidence from an external store and passes it to the generator. This grounding reduces hallucinations (fluent but unsupported statements) and makes each claim traceable to specific items of evidence.

Retrieval is usually performed with approximate nearest-neighbor (ANN) search in a vector space, often using an HNSW index (Hierarchical Navigable Small World; Malkov & Yashunin, 2020), a layered graph structure that efficiently finds approximate nearest neighbors. This enables sub-second searches for over a billion vectors. GenDFIR (Loumachi et al., 2025) applies RAG to cyber-incident timelines, while Multi-RAG (Mao et al., 2025) extends retrieval across video, audio, and text for multimodal analysis (Appendix A.4).

Two limitations remain. **First,** most forensic RAG systems still operate primarily on text and cannot search directly over rich video representations, which is limiting when much of the evidence is visual. **Second**, multimodal systems that include video often target general tasks such as captioning or summarization, rather than structured forensic reconstruction, where outputs must follow a consistent narrative and be fully evidence-grounded.

2.6 Research Gap and Contribution

The areas above have all made progress, but mostly in isolation: open-vocabulary search, temporal understanding, event anticipation, causal anomaly reasoning, and multimodal or forensic RAG. What is still missing in the open literature is a single, reproducible architecture that brings these strands together.

Specifically, there is little work on systems that combine:

i. Event anticipation across multiple domains using a shared semantic backbone.
ii. causal explanations that are produced at prediction time, not only after detection.
iii. automated multimodal forensic reconstruction grounded in the same vector store, evaluated across different environments such as surveillance, sports, and traffic.

This project addresses this gap by building an end-to-end pipeline and measuring cross-domain transfer.

3. Engineering Process

3.1 Phase A Progress

Phase A turns the OmniSight concept into a complete, testable design. The work followed a deliberate order: we built the literature foundation (Chapter 2) and identified the research gap, derived the system requirements from domain and literature analysis (§3.2), designed the architecture (§3.3), selected datasets matching each domain and capability (§3.4), and specified the six core algorithms of the pipeline (§3.5) — checking that each stage's output is a valid input to the next. We then defined measurable success metrics and a test plan (Chapter 4) before any code is written.

Phase A deliverables

- A literature review across five research areas, with an explicit research gap and contribution statement.
- Functional and non-functional requirements, the latter organized under the ISO 25010 quality model.
- A system design And architecture: a use-case model, a containerized deployment architecture, and the MCP integration design.
- Five public datasets covering all three target domains, each mapped to the algorithm and success metric it serves.
- Six core algorithms, described in plain language in the body, specified as pseudocode in Appendix B, and each with a flow diagram.

- A quantitative evaluation plan (SM-1–SM-6) and a functional test plan (TC-1–TC-10).

Phase B will implement this design and run the Predictive Forecasting Module's cross-domain transfer evaluation.

Client interface. Throughout Phase A the team worked with the project advisor, Dr. Reuven Cohen, as the client/stakeholder. Progress was reviewed in regular meetings where the key decisions - the research gap, the algorithm pipeline, dataset selection, and the evaluation plan - were presented, discussed, and approved before being finalized in this book. This working relationship continues in Phase B, with the advisor reviewing implementation milestones and test results.

3.2 Requirements

3.2.1 Requirements elicitation

The requirements below were derived from four sources.

Analysis of current practice. We examined how video-analytics systems are used in operational settings today across the security, sports, and traffic sectors. In current practice, footage is indexed by predefined categories, review is retrospective rather than anticipatory, deployments are tuned to a single domain, and multi-camera reconstruction is done manually. We concluded that the gap is not detection accuracy but the absence of an integrated pipeline that searches by meaning, anticipates events, and assembles evidence automatically — and this set the functional scope of the requirements below.

Literature analysis. Chapter 2 established what is technically feasible and where current methods fall short, which we translated into functional needs.

Stakeholder consultation. Regular consultations with the project advisor, Dr. Reuven Cohen, as project stakeholder, refined scope and priorities and validated each requirement.

Operational scenario analysis. Real-world scenarios in the five target datasets (§3.4) surfaced constraints such as real-time latency and multi-camera coverage, which shaped the non-functional requirements.

3.2.2 Functional Requirements (FR)

The following table enumerates the core functional requirements of the OmniSight platform, spanning video ingestion, AI processing, semantic search, predictive forecasting, agentic verification, forensic reporting, enterprise interoperability, and fault tolerance.

ID	Functional Requirement
FR-1a	The system shall accept video input from live camera feeds and from uploaded video files.
FR-1b	The system shall identify and keep the most informative frames by detecting movement.
FR-2	The system shall analyze each accepted video segment to produce a searchable representation of its visual content and its temporal changes.

FR-3	The system shall store each analyzed segment along with its associated details (time, source camera, and a representative image) for later retrieval.
FR-4	The system shall allow users to search for the stored video using natural-language queries and shall return a ranked list of the most relevant moments based on both visual content and temporal changes.
FR-5	The system shall analyze recent activity over a configurable time window and predict likely upcoming events, including the event type, the estimated time until the event, and a confidence level.
FR-6	The system shall evaluate each predicted event and decide whether to raise an alert, so that only alerts judged credible are presented to an operator.
FR-7	The system shall generate a structured incident report that summarizes the relevant evidence for a selected event, including the associated video segments and timeline.
FR-8	The system shall allow authorized external software systems to use its search, analysis, prediction, and live-alert capabilities.
FR-9	The system shall allow to search for videos, view live alerts, and review incident reports.
FR-10	The system shall authenticate users before accessing any system feature.

3.2.3 Non-Functional Requirements (NFR)

Non-functional requirements are organized according to the ISO 25010 software quality model.

ID	Non-Functional Requirement
Performance efficiency	
NFR-1	From the moment a video frame is captured, it must be ready to search in < 5 seconds.
NFR-2	When a user runs a search, results must appear in < 3 seconds.
NFR-3	Any forecast or prediction the system generates must be ready in < 2 seconds.
NFR-4	The system must handle ≥ 10 live video feeds at the same time without slowing down.
Reliability	
NFR-5	If any part of the system fails or crashes, the amount of video data or stored information that is lost = 0. Nothing is allowed to be lost.
Maintainability	
NFR-6a	Each major part of the system can be updated or replaced on its own, without affecting any of the other parts.
NFR-6b	New types of predictions can be added to the system without changing any existing parts
Security	
NFR-7	100% of API endpoints and MCP tools reject any request lacking a valid token or sufficient role — the number of resources reachable without authorization = 0.
NFR-8	100% of client and external-agent traffic is encrypted with TLS 1.2+ (AES-256); bytes of video, query, or alert data sent in plaintext = 0.

NFR-9	Authentication tokens expire after ≤ 15 minutes of inactivity, and an account is locked after ≥ 5 consecutive failed login attempts.

3.3 System Design

3.3.1 Use Case

The use-case model has three actors.

Users (operators and analysts) upload video (Upload Video), search it in plain language (Perform Semantic Search), open the live-feed and results screen (View Dashboard), receive alerts for imminent events (Receive Predictive Alert), confirm or dismiss them (Confirm or Dismiss Alert), and build multi-camera incident reports (Generate Forensic Dossier).

System Administrators manage accounts and permissions (Manage Users and Roles), register and configure cameras (Configure Camera), monitor status (Monitor System Health), and set footage retention (Manage Retention Policy).

External AI Agents reach the same capabilities over MCP: semantic search (Search Video Semantics), temporal action analysis (Analyze Temporal Action), a continuous alert feed (Live Alert Stream), and forecast retrieval (Get Predictive Alerts).

Authenticate is included by every user and administrator action, and Invoke MCP Tool by every agent action, so no request proceeds without verification. Generate Forensic Dossier extends Confirm or Dismiss Alert, since a confirmed alert may optionally trigger dossier generation.

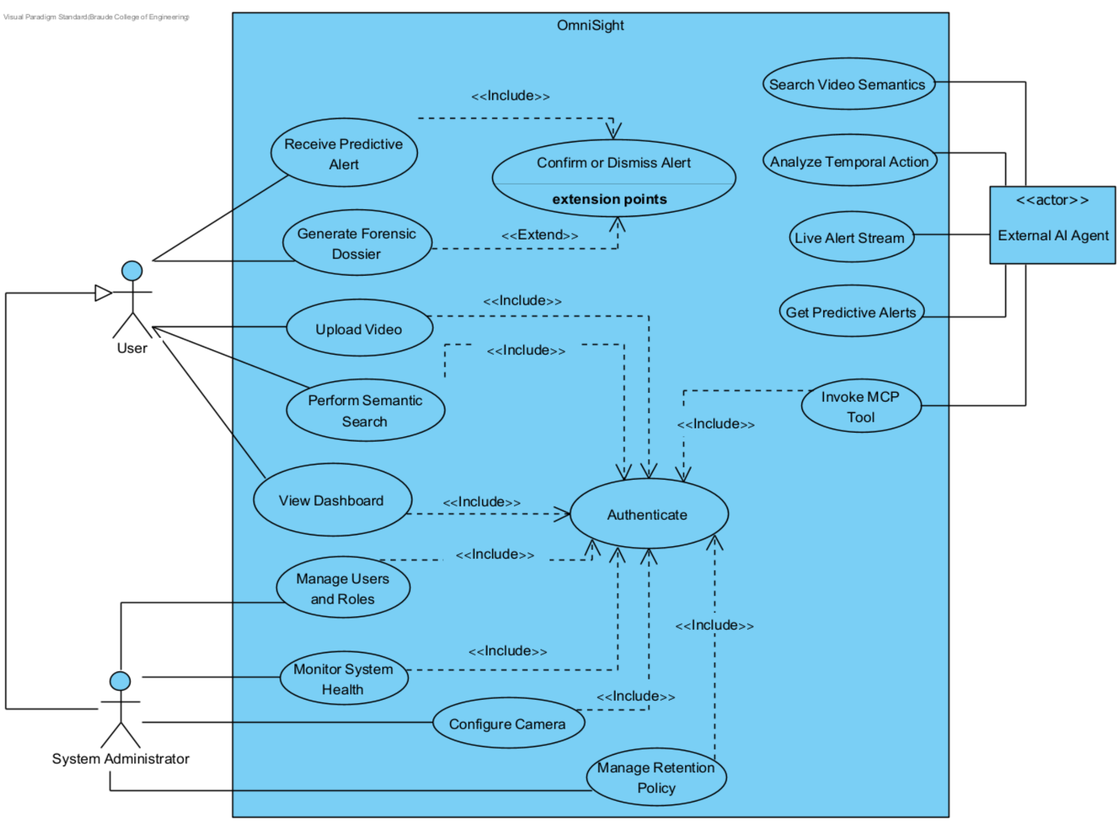


Figure 1 — OmniSight use-case model: user, administrator, and external-agent capabilities, with mandatory authentication and an optional dossier-generation flow.

3.3.2 High-level Deployment Architecture

OmniSight is deployed as **containerized microservices** orchestrated by **Docker Compose** (a tool that defines and runs multi-container applications from a single configuration). Every component runs in its own container on a shared internal network, keeping the system portable and letting individual services scale or restart independently.

The deployment is organized into three tiers.

- The **Frontend Tier** serves the React Dashboard through an Nginx container, the entry point for human operators over HTTPS.
- The **Backend Tier** contains four microservices: the Ingestion Engine (frame extraction via OpenCV/FFmpeg), the AI Workers (SigLIP 2 and InternVideo2 on GPU, horizontally scalable), the PFM + Verification service (temporal transformer plus VLM reasoning), and the Query Service (FastAPI for REST, FastMCP for MCP).
- The **Data Tier** holds Qdrant (vector database for embeddings) and PostgreSQL (metadata and alert logs).

Between the backend services sits **RabbitMQ**, a message broker that decouples ingestion from inference. Frames are published to work queues and consumed asynchronously by the AI Workers, with a dead-letter queue catching failures for retry.

Two external entry points exist: operators reach the dashboard over HTTPS, while external AI agents connect directly to the Query Service over MCP/JSON-RPC.

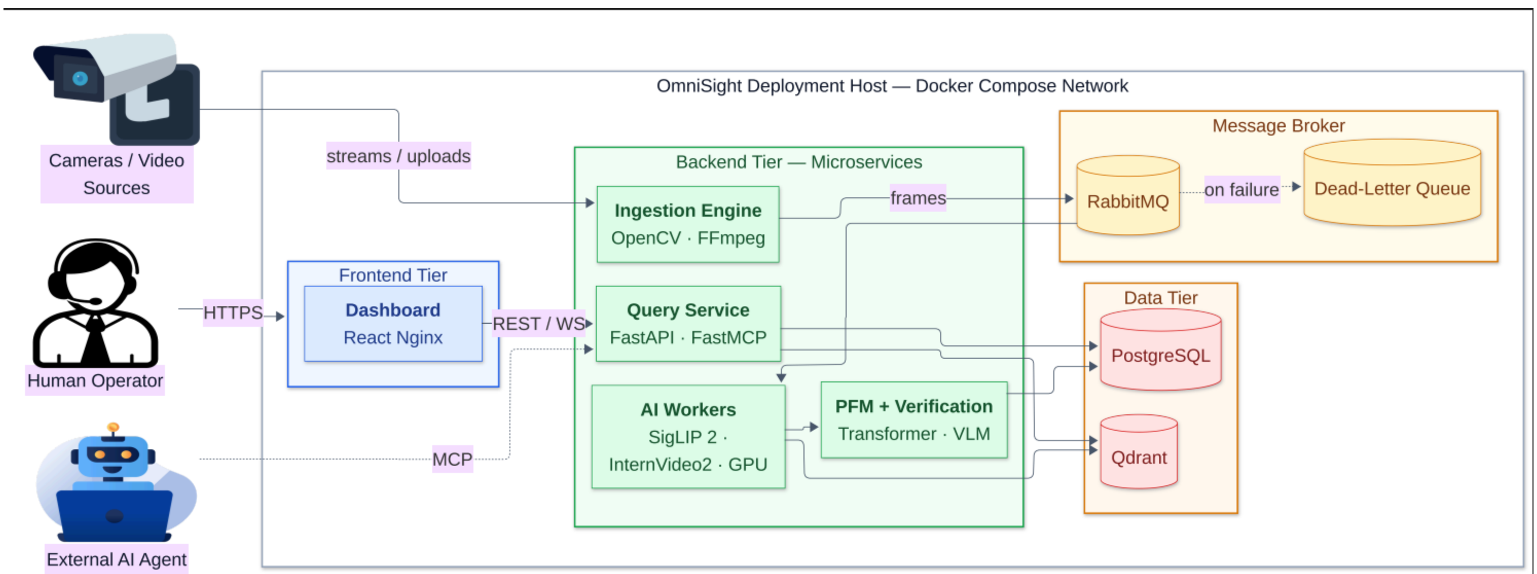


Figure 2 — High-level deployment architecture: containerized microservices across frontend, backend, and data tiers, decoupled by a RabbitMQ message broker.

3.3.3 MCP Server Integration

OmniSight makes its capabilities available to external AI agents through a Model Context Protocol (MCP) server, built with FastMCP. MCP is a standard way for an AI agent to call another system's functions directly, so a security, stadium, or traffic agent can use OmniSight without any custom integration code.

Each agent connects through its own MCP client and keeps an ongoing session with the server. When the session starts, the server tells the agent which capabilities it offers. OmniSight offers four capabilities:

Three callable tools including search video semantics, analyze temporal action, and get predictive alerts , and one live data stream, a live alert stream.

 Each tool call is forwarded to the backend component that handles it: the vector database, the Predictive Forecasting Module, or the Agentic Verification Layer, and verified alerts are pushed back to the agent over the same connection as they happen.

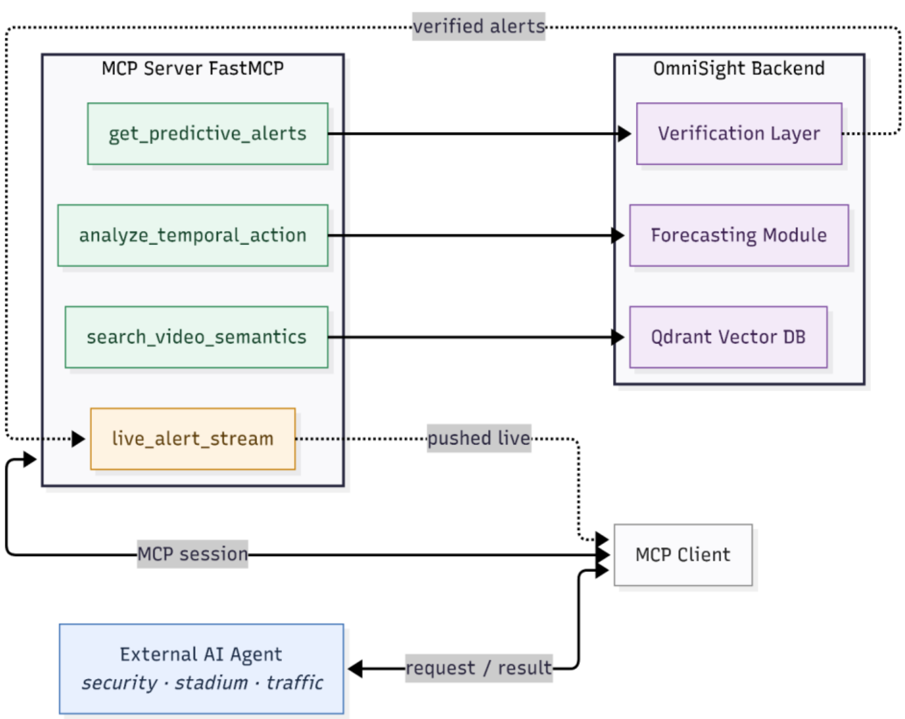


Figure 3 — MCP server integration: three callable tools and one live alert stream exposed to an external AI agent over a persistent session.

3.3.4 Runtime Activity Flow

A frame is first checked for motion: still frames are discarded, and active ones are encoded twice in parallel (once for scene content, once for short-term motion) and stored so the footage becomes searchable. The system then forecasts whether an incident is imminent; low-confidence cases stop quietly, while high-confidence ones raise an alert that passes through a verification cascade ,a sequence of checks that removes false alarms, before reaching the operator. The operator dismisses or confirms it, and only a confirmed alert triggers the final step — assembling a forensic dossier, a cited summary of the event for later review.

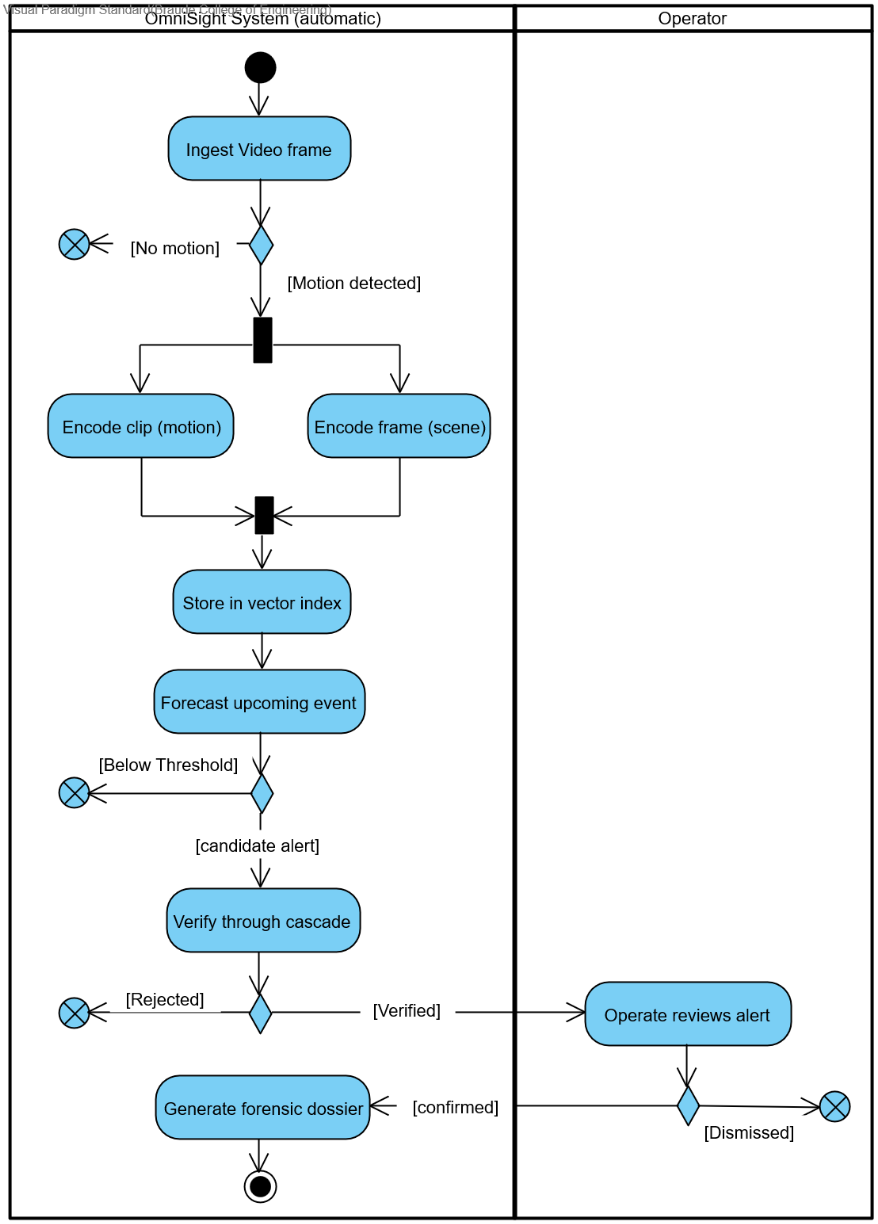


Figure 4 — OmniSight runtime activity flow: motion gating, parallel dual-stream encoding and indexing, event forecasting, verification, and operator-confirmed forensic synthesis.

3.4 Datasets

OmniSight will be trained and evaluated on five publicly available datasets covering its three target domains – surveillance, sports, and traffic. This spread is intentional. The Predictive Forecasting Module uses a shared temporal backbone with per-domain adapter heads. Hence, evaluation requires datasets that differ in viewpoint, event type, and visual regime, yet each carries annotations rich enough for training and measurement.

- **VIRAT (Oh et al., 2011)** - Surveillance. Stationary outdoor camera footage with object tracks and event annotations, primary dataset for ingestion, frame selection, embedding generation, and semantic-search evaluation.
- **UCF-Crime (Sultani et al., 2018) -** Surveillance. 1,900 untrimmed surveillance videos across 13 anomaly classes. will be used to train the PFM surveillance adapter and to evaluate agentic verification and false-positive reduction. Precursor windows will be derived semi-automatically, as the standard release provides video-level labels.
- **SoccerNet (Giancola et al., 2018)** - Sports. Broadcast soccer matches with dense timestamped events (goals, fouls, cards). will train the sports adapter and support cross-domain transfer evaluation.
- **DoTA (Yao et al., 2022)** - Traffic. 4,677 ego-centric dash-cam videos with anomaly categories and explicit start/end times. Its precursor–anomaly structure supports time-to-event and early-warning measurement and cross-domain generalization.
- **AI City Challenge (2025)** - Traffic. Multi-camera traffic-safety footage capturing incidents from synchronized viewpoints. The only dataset with overlapping multi-camera coverage, intended to validate cross-camera correlation in RAG forensic-dossier synthesis. Together, these datasets will allow OmniSight to be evaluated across all three target domains, with the cross-domain transfer central to the Predictive Forecasting Module measured on genuinely distinct visual regimes rather than a single homogeneous corpus.

3.5 Algorithms

This section presents the six core algorithms of the OmniSight pipeline in plain steps. Where a step relies on a specific pretrained model, the model is named (SigLIP 2, InternVideo2, or Qwen2.5-VL). All parameters and notation are collected in Appendix B.

3.5.1 Algorithm 1 – ROI-Aware Spatial-Motion Sampling

Goal: forward only the frames with meaningful motion, so static scenes are not processed unnecessarily.

1. For each camera, keep a short rolling buffer of recent frames, and wait for an initial warm-up so a full clip is always available.
2. For each new frame, compare it with an earlier buffered frame to obtain a map of what changed in the scene.
3. Divide that map into a grid of cells and count how many cells show significant motion.
4. If enough cells are active, admit the frame; otherwise, discard it.
5. To avoid losing static scenes, also admit a frame at a fixed interval even when no motion is detected, so parked cars or empty areas remain represented in the index.
6. When a frame is admitted, take the recent clip ending at the current moment and forward it together with the frame.

[Pseudocode: Appendix B.1]

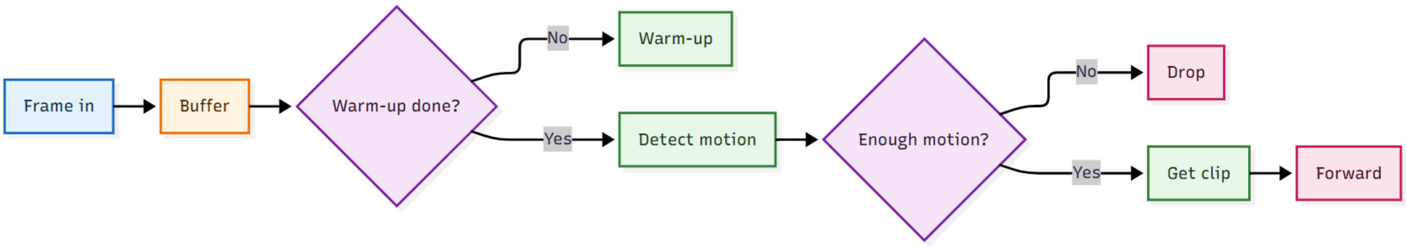


Figure 5 — ROI-aware spatial-motion sampling: per-camera buffering, grid-based motion gating, and causal clip forwarding.

3.5.2 Algorithm 2 – Dual-Stream Embedding Generation

Goal: capture both what a scene looks like and what is happening over time.

1. Take each admitted frame-and-clip pair produced by Algorithm 1.
2. Encode the single frame with **SigLIP 2** to capture the static scene.
3. In parallel, encode the clip with **InternVideo2** to capture motion and temporal context, first reducing the clip to the fixed number of evenly spaced frames the model expects.
4. Normalize both vectors to a common length so they can be compared fairly later.
5. Store both vectors in the vector database **(Qdrant)** with shared metadata — camera, time, and a common identifier — so the frame-level and clip-level entries can be matched during retrieval.

[Pseudocode: Appendix B.2]

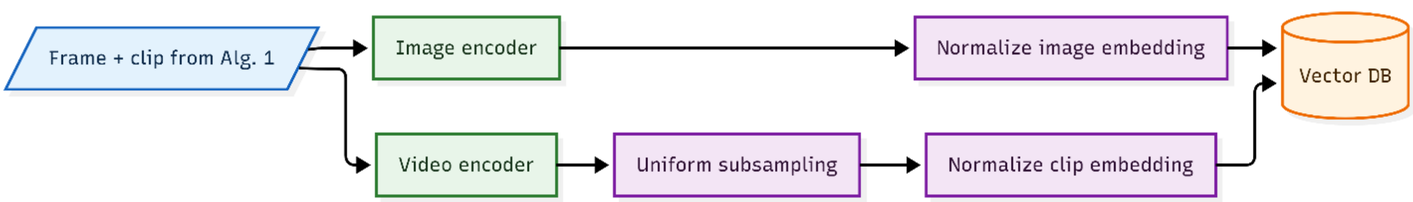


Figure 6 — Dual-stream embedding generation: parallel frame and clip encoders writing to a shared vector store.

3.5.3 Algorithm 3 - Dual-Model Score Fusion for Semantic Retrieval

Goal: return strong results whether the query describes a static scene or an action.

1. Pass the query to a small classifier that decides how much weight to give the scene model versus the action model.
2. Encode the query text once with the **SigLIP 2** text encoder and once with the **InternVideo2** text encoder.
3. Search each model's index separately, producing two ranked lists of candidates.
4. Merge the two lists by how highly each item ranks in each one (Reciprocal Rank Fusion), rather than by raw scores, because the two models' scores are not on the same scale.
5. Weight each list by the classifier's weights and return the combined top results.

[Pseudocode: Appendix B.3](#)

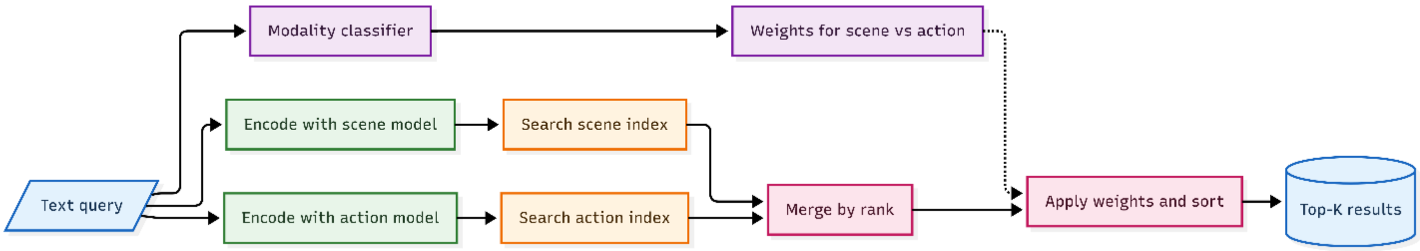


Figure 7 — Dual-model score fusion: modality-weighted reciprocal rank fusion over the frame and clip indexes.

3.5.4 Algorithm 4 - Predictive Forecasting Module (PFM)

The PFM is the project's main contribution: a single shared temporal backbone with one specialized head per domain.

3.5.4.1 Inference

Goal: anticipate an event from the recent stream of clip representations before it happens.

1. Collect a sliding window of the most recent clip embeddings (produced by InternVideo2 in Algorithm 2), spanning roughly half a minute to a minute.
2. Pass the window through the shared temporal backbone, which models long-range patterns across the whole window.
3. Route the backbone's output to the head for the current domain — surveillance, sports, or traffic.
4. Have that head produce the predicted event type together with an estimate of how long until the event occurs.
5. Derive a confidence value from how sharply the prediction concentrates on a single outcome.
6. If both the event likelihood and the confidence pass the domain's threshold, raise a candidate alert.

[Pseudocode: Appendix B.4](#)

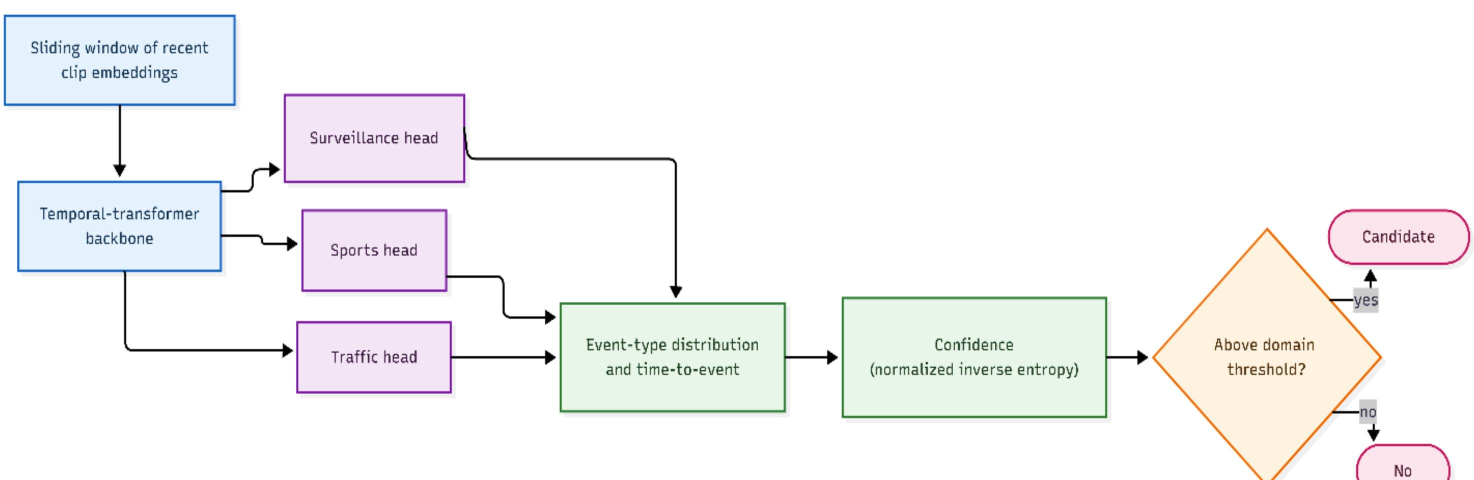


Figure 8 — PFM inference: a shared temporal backbone feeding per-domain prediction heads.

3.5.4.2 Training

Goal: train one shared backbone that serves all three domains, with each head specializing.

6. Train each domain head on its own dataset: surveillance on UCF-Crime, sports on SoccerNet, traffic on DoTA.
7. Jointly optimize the shared backbone and all three heads, balancing each domain's contribution and combining event classification with time-to-event estimation.
8. Set each domain's alert threshold on a held-out validation split.

Pseudocode and training objective: Appendix B.4

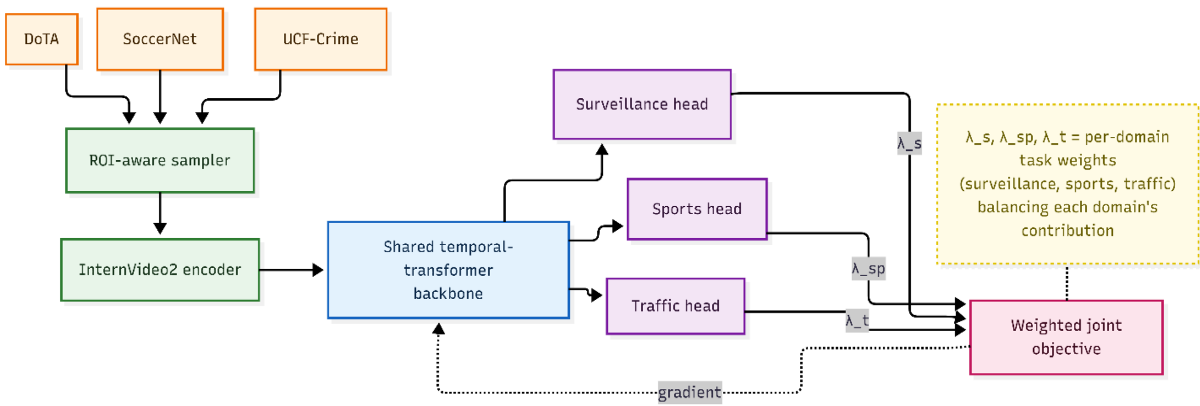


Figure 9 — PFM joint training: per-domain datasets feeding a shared backbone with specialized heads.

3.5.4.3 Cross-Domain Transfer Protocol

Hypothesis: pre-training the shared backbone on two domains yields temporal representations that transfer to an unseen third domain better than training on that domain alone.

1. Train the backbone and heads jointly on two of the three domains.
2. Freeze the backbone and remove the two trained heads.
3. Attach a fresh head for the third domain and fine-tune only that head on a small slice of its data.
4. Measure how often the correct event appears among the top predictions on the third domain's test set.
5. Compare this against a model trained from scratch on the same small slice.
6. Repeat for all three choices of held-out domain.

Formal protocol: Appendix B.4

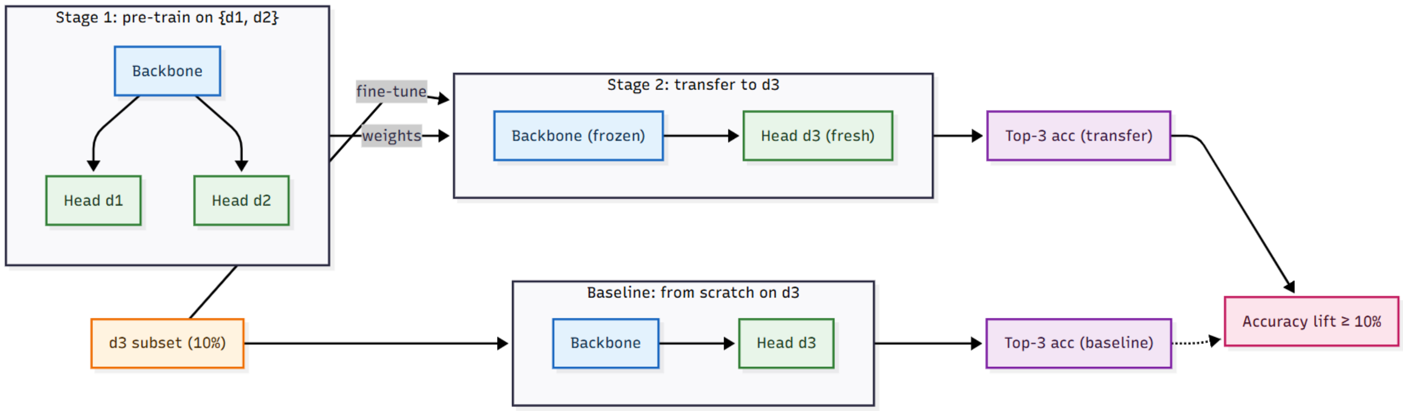


Figure 10 — Cross-domain transfer protocol: pre-train on two domains, freeze the backbone, fine-tune a fresh head on the third.

3.5.5 Algorithm 5 - Four-Tier Agentic Verification Pipeline

Goal: turn a continuous stream of candidates into a few trustworthy alerts by filtering from cheapest to most expensive.

1. **Tier 1** — quickly check whether a clip is even similar to known anomaly concepts, and drop it if not.
2. **Tier 2** — run the PFM and keep only clips whose predicted event and confidence are high enough.
3. **Tier 3** — send a few sampled frames to **Qwen2.5-VL** with a structured prompt, ask it to reason about the cause and return a verdict, and drop clips it rejects.
4. **Tier 4** — pass only the survivors to the human operator, with the reasoning trace and an estimated severity attached.

Pseudocode: Appendix B.5

3.5.6 Algorithm 6 - Multimodal RAG Forensic Synthesis

Goal: after an operator confirms an incident, assemble an evidence-grounded report in which every claim is cited.

1. Summarize the verification reasoning into a short query and pair it with the clip that triggered the alert.
2. Restrict retrieval to a time window around the incident.
3. Encode the query with the **SigLIP 2** and **InternVideo2** text encoders and retrieve related evidence from both the frame index and the clip index.
4. Re-rank the evidence by a combination of similarity, closeness in time to the incident, and camera overlap with the incident camera.
5. Group the surviving evidence into short time bins and keep items from cameras whose fields of view overlap.
6. Tag every retained item with a citation identifier.
7. Have **Qwen2.5-VL** write the report, requiring it to cite every claim and to introduce no facts that are not in the supplied evidence.

Pseudocode: Appendix B.6

3.6 Expected Challenges

The main difficulties we expect during development, and how we plan to handle each.

- **Predicting events before they happen:** The system must warn about an event before it happens, but the early signs are faint and easy to miss; if we look at the wrong slice of time beforehand, the model reacts after the fact instead of predicting it.
  Mitigation: train the model on easy cases first, then harder ones (curriculum learning); test different "lead-up" window lengths to find the best; and combine several models so their mistakes cancel out.
- **Making one model work across very different settings:** One shared model must handle surveillance, sports, and traffic, but the build-up to an event looks completely different in each (a crime, a goal, a crash), so a model that learns one may fail in another.
  Mitigation: train the shared core on all three together, add a small, specialized part per setting, and test on a setting it has never seen to confirm its skills carry over.
- **Keeping up with live video in real time:** Every frame passes through several heavy steps: two AI models that turn it into searchable data, saving to the search database, and the prediction model, all within 5 seconds per frame on limited student-grade hardware.
  Mitigation: process only moving frames, spread the work across many machines with a job queue so nothing sits idle, and handle frames in groups to use the GPU efficiently.
- **The explanation step is slow:** The part that writes a human-readable explanation (the vision-language model) is by far the slowest; running it on every frame would make real-time operation impossible.
  Mitigation: run it only when faster earlier steps flag something worth explaining, reuse results already computed, and use smaller, faster versions of the model. If it still cannot keep up, fall back to alerts from the prediction model alone.
- **No dataset has the labels we need:** No dataset has everything: video from all three settings with the moments just before each event marked. We use UCF-Crime (surveillance), SoccerNet (sports), DoTA (traffic), and VIRAT (search), but each was built for a different purpose, and none marks the "lead-up" moments our predictor needs, so we create them ourselves. Only AI City Challenge films with several cameras at once, limiting our multi-camera tests.
  Mitigation: combine the four datasets, mark the lead-up moments mostly automatically by working backward from each known event, and check a sample by hand.

4. Tests and Metrics

4.1 Success Metrics and Acceptance Criteria

Each design goal maps to a measurable criterion, tested on the dataset best suited to it (see §3.4).

- **SM-1 – Semantic Search Quality:** The correct video clip appears in the top 5 results for at least 85% of natural-language queries, with a clear lift on action queries over a frame-only baseline.
  How to measure: Run a labeled query set on VIRAT (clips with known matches), count how often the right clip lands in the top 5, and compare against the frame-only baseline.

- **SM-2 – Prediction Accuracy:** The real event is among the model's top 3 predictions in at least 70% of cases, per domain.
  How to measure: On held-out clips from UCF-Crime (surveillance), SoccerNet (sports), and DoTA (traffic), check whether the annotated event appears in the model's three most likely predictions.

- **SM-3 – Early Warning Time:** Warnings fire at least 15 seconds before the event begins – enough time for human reaction.
  How to measure: On DoTA, which provides exact event start times, average the gap between each correct warning and the annotated event starts.

- **SM-4 – False Alarm Rate:** No more than 20% of warnings are false, per domain.
  How to measure: On UCF-Crime (including its normal footage), divide warnings that do not match an annotated event by total warnings raised.

- **SM-5 – Cross-Domain Learning:** Pre-training on two domains lifts third-domain accuracy by at least 10% over training on that domain alone.
  How to measure: Train the backbone on two of {UCF-Crime, SoccerNet, DoTA}, fine-tune on the third, and compare to a from-scratch baseline. Repeat with each domain held out.

- **SM-6 – Forensic Report Generation:** The system produces a structured incident report for at least 2 multi-camera scenarios, combining evidence into a coherent timeline.
  How to measure: On AI City Challenge footage (the only multi-camera dataset), run the full retrieve-synthesize-generate pipeline and validate each report against a scenario checklist (correct cameras, right chronology, evidence-grounded claims).

4.2 Tests

Each test is traced to the functional requirement(s) it verifies. Every requirement is covered by at least one test.

ID	Objective (Requirement)	Test Steps	Expected Result
TC-1	Ingest from a live feed and an uploaded file at once, keeping only	1. Upload a valid MP4 via the ingest API.	Both sources decode without error; only frames containing motion are forwarded.

ID	Objective (Requirement)	Test Steps	Expected Result
	frames with motion (FR-1a, FR-1b).	2. Connect a live RTSP camera stream at the same time. 3. Run both sources for 30 seconds.	
TC-2	Produce a frame-level and a clip-level embedding for each admitted segment (FR-2).	1. Submit 30-second segments (MP4 and RTSP) to the embedding worker. 2. Confirm output from both SigLIP 2 (frame encoder) and InternVideo2 (clip encoder).	Each segment returns two L2-normalized embedding vectors with no error.
TC-3	Store each embedding together with its metadata in Qdrant (FR-3).	1. Insert a batch of frame and clip embeddings, with metadata, into both Qdrant collections. 2. Query Qdrant to retrieve the records and inspect the metadata.	Every embedding is retrievable with all metadata fields intact; no data is lost or corrupted.
TC-4	Search a large archive by plain-English query, returning fast and accurate results (FR-4).	1. Pre-load both collections with ≥ 100,000 embeddings from VIRAT (or equivalent labeled video). 2. Run natural-language queries covering static scenes and dynamic actions. 3. Measure query latency and recall@5 against ground-truth matches.	The correct clip appears in the top 5 results for **≥ 85%** of queries.
TC-5	Predict upcoming events accurately and early, keep false alarms bounded, and transfer to an unseen domain (FR-5).	1. Run PFM inference on 30s and 60s sliding windows over held-out clips from UCF-Crime, SoccerNet, and DoTA. 2. For transfer: train the shared backbone on two domains, freeze it, then k-shot fine-tune a fresh head on the third. 3. Compare against a from-scratch baseline; repeat with each domain held out.	Top-3 accuracy **≥ 70%**, early-warning lead **≥ 15 s**, false-positive rate **≤ 20%**, cross-domain transfer gain **≥ 10%**.
TC-6	Four-tier verification separates genuine anomalies from normal activity and cuts false positives versus PFM alone (FR-6).	1. Feed a mixed batch of labeled anomaly and benign clips (UCF-Crime) through all four tiers. 2. Record which clips are escalated or suppressed, and the tier at which suppression occurs. 3. Compare the false-positive rate before and after verification.	Anomaly clips pass all four tiers and reach the operator with a reasoning trace and severity score. Benign clips are suppressed at Tier 1, 2, or 3. Post-verification false-positive rate is measurably lower than the raw PFM rate.
TC-7	After operator confirmation, assemble an evidence-grounded dossier from multiple cameras (FR-7).	1. Using AI City Challenge footage (synchronized multi-camera), trigger RAG forensic synthesis for ≥ 2 incident scenarios, each spanning ≥ 2 overlapping cameras.	A structured dossier is produced per scenario; every event is timestamped.

ID	Objective (Requirement)	Test Steps	Expected Result
		2. Inspect each dossier for structure and cross-camera coherence.	Every factual claim is cited to a specific clip or frame, with no unsupported statements. Evidence from ≥ 2 cameras is present and the cross-camera timeline is consistent.
TC-8	Expose all capabilities to external AI agents through the MCP server, including real-time alerts (FR-8).	1. Connect a test AI agent to the MCP server. 2. Have it call semantic search, temporal action analysis, event prediction, and live-alert retrieval. 3. Validate each tool's JSON schema against the MCP specification. 4. Trigger a synthetic alert while the agent is connected.	All four tools are discoverable with valid schemas and well-formed responses. The agent receives the synthetic alert over the live stream within the expected latency.
TC-9	Render the search, alerts, and dossier views in the web dashboard (FR-9).	1. Enter and submit a natural-language search query. 2. Open the alerts view and check that existing alerts are listed. 3. Click a completed incident to open its dossier view.	All three views render with no JavaScript errors or blank states.
TC-10	Protect every endpoint with authentication and role-based authorization (FR-10).	Send requests to every protected API endpoint and MCP tool under three conditions: 1. Valid token with the correct role. 2. Valid token but an insufficient role. 3. No token, or an expired/invalid token.	No protected resource is accessible without valid authentication and sufficient authorization under any condition.

References

Academic References (APA, alphabetical)

1. Bai, S., et al. (2025). Qwen2.5-VL technical report. *arXiv.* https://arxiv.org/abs/2502.13923
2. Cao, C., Lu, Y., Wang, P., & Zhang, Y. (2023). A new comprehensive benchmark for semi-supervised video anomaly detection and anticipation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. https://arxiv.org/abs/2305.13611
3. Cheng, Y., Lin, Y.-H., Chen, M.-H., Yang, F.-E., & Lai, S.-H. (2026). VADER: Towards causal video anomaly understanding with relation-aware large language models. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. https://arxiv.org/abs/2511.07299
4. Felsen, P., Lucey, P., & Ganguly, S. (2017). What will happen next? Forecasting player moves in sports videos. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. https://openaccess.thecvf.com/content_ICCV_2017/papers/Felsen_What_Will_Happen_ICCV_2017_paper.pdf
5. Giancola, S., Amine, M., Dghaily, T., & Ghanem, B. (2018). SoccerNet: A scalable dataset for action spotting in soccer videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. https://openaccess.thecvf.com/content_cvpr_2018_workshops/papers/w34/Giancola_SoccerNet_A_Scalable_CVPR_2018_paper.pdf
6. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems (NeurIPS), 33*, 9459–9474. https://arxiv.org/abs/2005.11401
7. Loumachi, F. Y., Ghanem, M. C., & Ferrag, M. A. (2025). Advancing cyber incident timeline analysis through retrieval-augmented generation and large language models. *Computers, 14*(2), Article 67. https://doi.org/10.3390/computers14020067
8. Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 42*(4), 824–836. https://arxiv.org/abs/1603.09320
9. Mao, M., Perez-Cabarcas, M. M., Kallakuri, U., Waytowich, N. R., Lin, X., & Mohsenin, T. (2025). Multi-RAG: A multimodal retrieval-augmented generation system for adaptive video understanding. *arXiv.* https://arxiv.org/abs/2505.23990
10. Oh, S., Hoogs, A., Perera, A., et al. (2011). A large-scale benchmark dataset for event recognition in surveillance video. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. https://doi.org/10.1109/CVPR.2011.5995586

11. Radford, A., Kim, J. W., Hallacy, C., et al. (2021). Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*. https://arxiv.org/abs/2103.00020
12. Sultani, W., Chen, C., & Shah, M. (2018). Real-world anomaly detection in surveillance videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. https://arxiv.org/abs/1801.04264
13. Tschannen, M., et al. (2025). SigLIP 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features. *arXiv.* https://arxiv.org/abs/2502.14786
14. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS), 30*. https://arxiv.org/abs/1706.03762
15. Wang, Y., Li, K., Li, X., et al. (2024). InternVideo2: Scaling foundation models for multimodal video understanding. In *Proceedings of the European Conference on Computer Vision (ECCV)*. https://arxiv.org/abs/2403.15377
16. Yao, Y., Wang, X., Xu, M., et al. (2022). DoTA: Unsupervised detection of traffic anomaly in driving videos. *IEEE Transactions on Pattern Analysis and Machine Intelligence*. https://arxiv.org/abs/2004.03044

Non-Academic / Technical References

1. AI City Challenge. (2025). *Track 2: Multi-camera traffic safety analysis.* https://www.aicitychallenge.org
2. Anthropic. (2025). *Model Context Protocol specification.* https://modelcontextprotocol.io
3. Qdrant. (2025). *Vector similarity search engine — Documentation.* https://qdrant.tech/documentation

Appendix A: Representative Model Architectures

This appendix collects the detailed descriptions of the representative models surveyed in §2.2. Each entry expands on the architecture and training objectives referenced in the corresponding subsection of the literature review.

A.1 Contrastive Vision–Language Embedding (SigLIP 2)

SigLIP 2 (Tschannen et al., 2025) is among the strongest publicly available open-vocabulary models. It replaces CLIP's batch-wide softmax with a sigmoid loss that scores each image–text pair independently, improving training stability at large batch sizes. In addition to the contrastive objective, it adds captioning-based pretraining (a decoder generates captions and bounding-box references from image features), self-supervised dense-feature objectives, and refined data curation, producing tighter visual–textual alignment than earlier CLIP-style models. The architecture is a standard dual encoder: a Vision Transformer (ViT) — a model that splits an image into fixed-size patches and processes them with self-attention, the mechanism by which each patch attends to every other patch — encodes the image, while a Transformer text encoder maps the caption into the same space, and matches are scored by cosine similarity. SigLIP 2 reports consistent gains on zero-shot classification, image–text retrieval, transfer, open-vocabulary localization, and dense prediction.

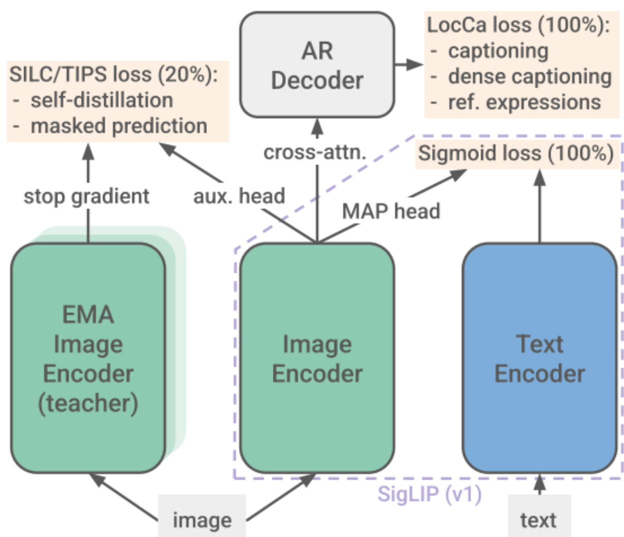


Figure 13 - SigLIP 2 architecture (Source: Tschannen et al., 2025).

A.2 Temporal Video Foundation Model (InternVideo2)

InternVideo2 (Wang et al., 2024) is a widely cited video foundation model trained with a progressive scheme that combines three objectives: masked video modeling, in which parts of the video are hidden, and the model reconstructs them; multimodal contrastive learning across video, text, and audio, which aligns matching triplets in a shared embedding space; and next-token prediction over interleaved video–language sequences—together these capture appearance, motion, semantics, and discourse-level structure in one backbone. The model encodes short clips through stacked spatio-temporal attention blocks — self-attention applied jointly across the spatial dimensions of each frame and across frames — producing clip-level embeddings that represent entire actions rather than isolated images. The authors report state-of-the-art or competitive results on more than sixty video benchmarks spanning action recognition, retrieval, video question answering, and temporal localization, making it a natural choice when motion sensitivity is required.

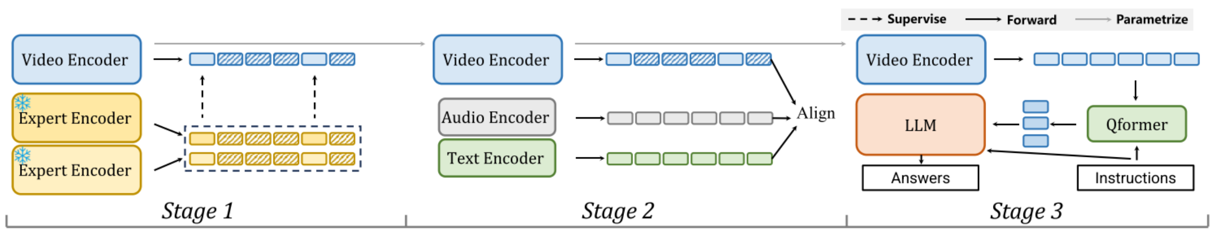


Figure 14 - InternVideo2 architecture (Source: Wang et al., 2024).

A.3 Generative Vision–Language Model (Qwen2.5-VL)

Qwen2.5-VL (Bai et al., 2025) is a large pretrained vision–language model that accepts images or short clips with natural-language prompts and produces grounded, free-form text. Unlike contrastive encoders such as CLIP and SigLIP 2, which emit fixed embedding vectors for matching and retrieval, Qwen2.5-VL is generative: it describes scenes, answers questions, follows instructions, performs spatial grounding (locating objects or regions referenced in the prompt), and produces multi-step reasoning chains. Architecturally, the model combines a ViT encoder with a large language model (LLM) decoder — a Transformer trained to generate text one token at a time, conditioned on prior context — linked through a learned MLP projection that maps visual features into the language model's input space. The vision encoder uses dynamic-resolution processing and window attention (self-attention restricted to local windows to reduce computational cost) to efficiently handle native-resolution input. At the same time, a multimodal rotary position embedding aligned to absolute time enables second-level event localization in videos lasting up to hours. The model is released openly in 3B, 7B, and 72B sizes and is among the strongest open VLMs on visual question answering, document understanding, and video benchmarks, making it a representative example of generative VLMs that serve as reasoning engines atop detection or retrieval components.

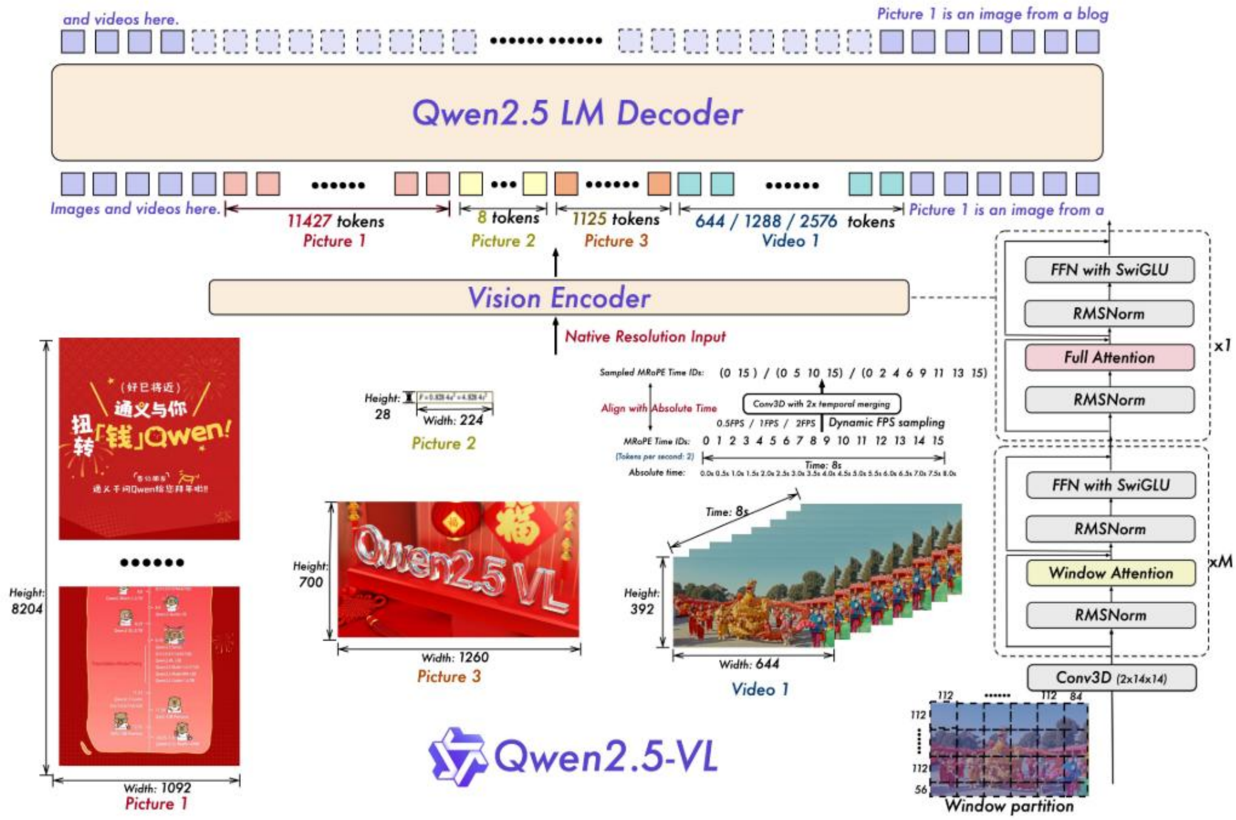


Figure 15 - Qwen2.5-VL architecture (Source: Bai et al., 2025).

A.4 Retrieval-Augmented Generation and ANN Indexing

Retrieval-Augmented Generation (Lewis et al., 2020) conditions a generative model on evidence retrieved on the fly from an external knowledge store. Retrieval is typically approximate nearest-neighbor (ANN) search in a dense vector space using indexes such as HNSW (Hierarchical Navigable Small World; Malkov & Yashunin, 2020), a graph-based index that achieves sub-second retrieval over billions of vectors. Grounding generation in retrieved content reduces hallucination, improves factual accuracy, and makes outputs traceable to their sources. Multimodal RAG generalizes this beyond text, retrieving images, clips, audio, or structured records alongside text and passing them to a VLM or multimodal LLM. This is particularly relevant when evidence is heterogeneous — for example, surveillance footage interleaved with sensor logs — and the desired output is a structured, evidence-grounded narrative.

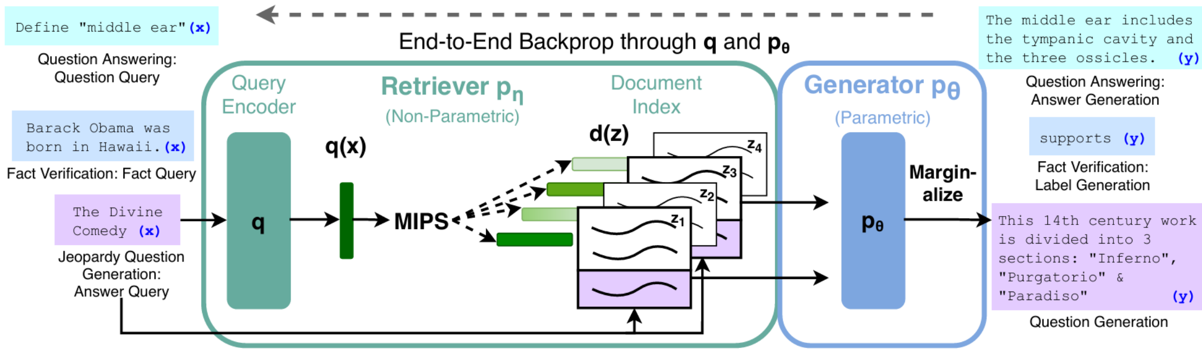


Figure 16 - Retrieval-Augmented Generation pipeline (Source: Lewis et al., 2020).

Appendix B — Algorithm Pseudocode

This appendix gives the formal pseudocode for the six algorithms described verbally in §3.5. Notation is local to each algorithm; the cross-stage data contracts (the inputs each stage receives and the outputs it forwards) are stated in the Input/Output headers.

Algorithm 1 — ROI-Aware Spatial-Motion Sampling

```
Input:  f_t (current frame); grid size G; threshold τ ∈ [0,255];
        min. active cells k; per-camera buffer Buffer_cam; stride Δ
Output: forwarded payload (f_t, W) | drop | warmup

Buffer_cam.append(f_t)
if Buffer_cam.duration < 2Δ then return warmup
diff  ← | grayscale(f_t) - grayscale(f_{t-Δ}) |
cells  ← partition(diff, G x G)
active ← count( c ∈ cells : mean(c) > τ )
if active ≥ k then
    W ← Buffer_cam.window([ t - 2Δ , t ])      // causal clip
    forward(f_t, W)
else
    drop(f_t)
```

Algorithm 2 — Dual-Stream Embedding Generation

```
Input:  (f_t, W) from Alg. 1; camera_id; timestamp; domain
Output: L2-normalized embeddings stored in Qdrant

e_f ← SigLIP2_image(f_t);       e_f ← e_f / ||e_f||   // 1152-D frame emb., L2
W_s ← uniform_subsample(W, 8 frames @ 224 x 224)
e_c ← InternVideo2_clip(W_s);    e_c ← e_c / ||e_c||   // 768-D clip emb., L2
meta ← { timestamp, camera_id, domain,
         frame_id = hash(camera_id, timestamp), thumb(f_t) }
Qdrant.upsert(DB_frame, e_f, meta)
Qdrant.upsert(DB_clip,  e_c, meta)
```

Algorithm 3 — Dual-Model Score Fusion via Reciprocal Rank Fusion

```
Input:  query q; modality classifier C; top-K; RRF constant k0 = 60
Output: ranked top-K result list R

(w_f, w_c) ← C(q)                    // softmax weights, w_f + w_c = 1
v_f ← SigLIP2_text(q);   v_c ← InternVideo2_text(q)
cand_f ← HNSW(DB_frame, v_f, K);  cand_c ← HNSW(DB_clip, v_c, K)
M ← unique frame_ids in cand_f ∪ cand_c
for each m ∈ M do
    r_f ← rank of m in cand_f    (∞ if absent)
    r_c ← rank of m in cand_c    (∞ if absent)
    score(m) ← w_f / (k0 + r_f) + w_c / (k0 + r_c)
return top-K of sort(M by score, descending)
```

Algorithm 4 — PFM Inference (Shared Backbone, Domain Head)

```
Input:  window W = [ e_{t-n+1}, ..., e_t ]; domain d; threshold θ_d
Output: (p, Δt, κ); optional candidate_alert

h        ← Backbone_Transformer(W)        // shared across domains
(p, Δt)  ← Head_d(h)                      // Δt measured forward from t
κ        ← 1 - H(p) / log(|C_d|)          // normalized confidence (inverse entropy)
if max(p) ≥ θ_d then emit candidate_alert(p, Δt, κ, W)
return (p, Δt, κ)

--- Joint training objective ---
L = Σ_d  λ_d * [ CE(p_d, y_d) + α * MSE(Δt_d, τ_d) ],   d ∈ {surv., sports, traffic}

--- Cross-domain transfer test ---
train on {d1, d2};  freeze backbone;  fine-tune fresh Head_{d3} on k-shot subset of d3.
```

Algorithm 5 — Four-Tier Agentic Verification

```
Input:  stream of (e_c, W, camera_id, t); anomaly concept vectors V_anom;
        thresholds σ1, θ_d, σ_κ
Output: verified_alert | suppression reason

-- Tier 1 . Similarity Trigger --
if max_{v ∈ V_anom} cos(e_c, v) < σ1 then return suppress("trigger")

-- Tier 2 . PFM Predictive Scoring --
(p, Δt, κ) ← PFM(W, d)                    // Algorithm 4
if max(p) < θ_d or κ < σ_κ then return suppress("pfm")

-- Tier 3 . VLM Causal Verification --
resp ← Qwen2.5-VL( uniform_sample(W, 8), COC_PROMPT, json_mode )
if not resp.verdict then return suppress("vlm", resp.coc_trace)

-- Tier 4 . Operator Escalation --
push_to_dashboard({ p, Δt, κ, coc_trace = resp.coc_trace,
                 severity = resp.severity })
return verified_alert
```

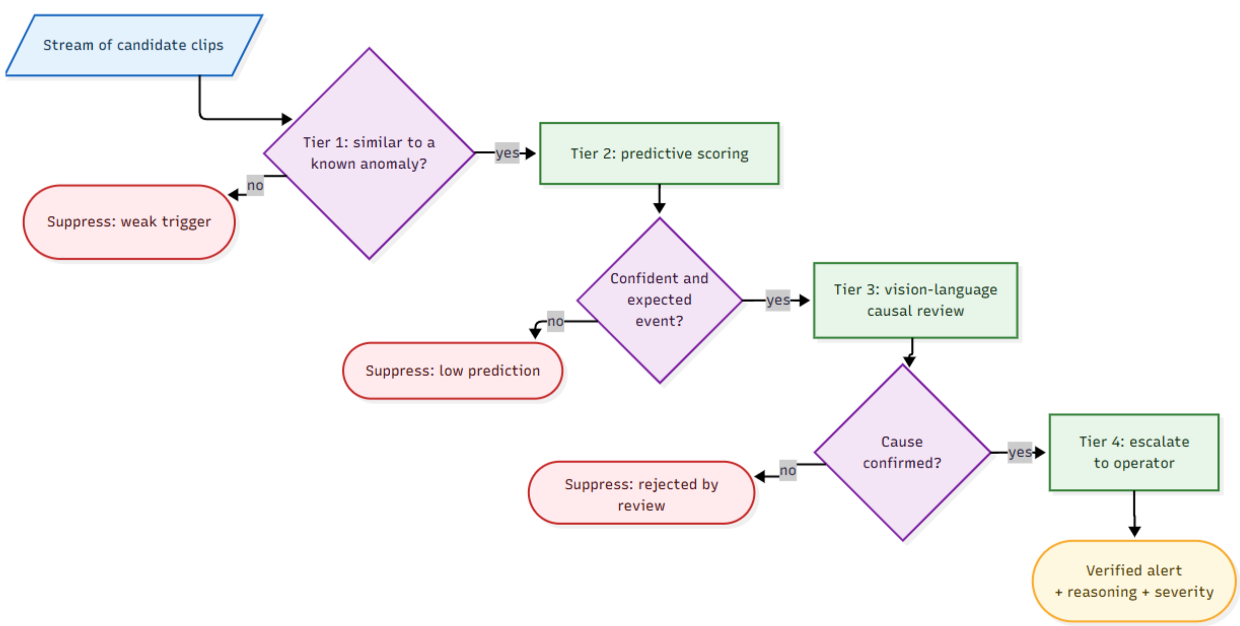


Figure 11 — Four-tier agentic verification cascade, from a cheap similarity trigger to operator escalation.

Algorithm 6 — Multimodal RAG Forensic Synthesis

```
Input:  incident i = (t, camera_id, coc_trace, e_c*); retrieval scope R; K1, K2
Output: forensic dossier D with inline citations

1:  Q_text ← summarize(coc_trace)
2:  Q_f ← SigLIP2_text(Q_text);   Q_c □ InternVideo2_text(Q_text)
3:  filter ← { timestamp ∈ [ t - R , t + R ] }
4:  E <- HNSW(DB_frame, Q_f, K1, filter)
       ∪ HNSW(DB_clip,  Q_c, K2, filter)
       ∪ HNSW(DB_clip,  e_c*, K2, filter)
5:  for x ∈ E do
6:     score(x) ← λ1*sim(x) + λ2*temporal_proximity(x, t)
               + λ3*spatial_overlap(x.camera_id, i.camera_id)
7:  E ← top-N of sort(E by score, descending)
8:  bins ← group(E by floor(x.timestamp / 2s))
9:  E ← keep items from FOV-overlapping cameras within each bin
10: for x ∈ E do x.cite_id □ format(x.camera_id, x.timestamp)
11: D ← LLM.synthesize(dossier_template, evidence = E,
      prompt = "Cite every claim by [cite_id]; introduce no facts not in evidence.")
12: return D
```

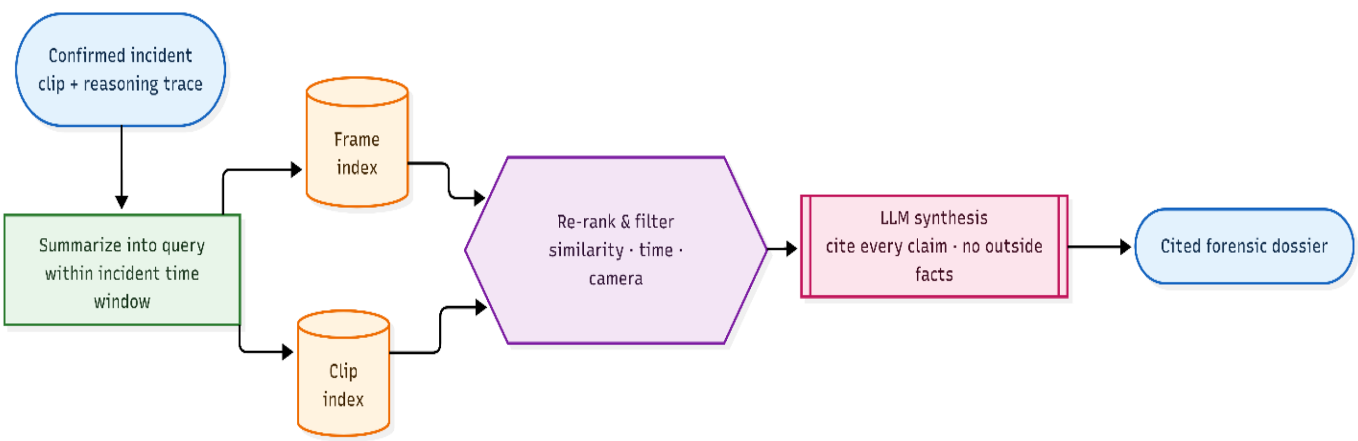


Figure 12 — Multimodal RAG forensic synthesis: scoped retrieval, cross-camera correlation, and cited dossier generation.

Appendix C - Glossary of terms

This appendix defines the technical terms and acronyms used throughout the book. Terms are listed alphabetically.

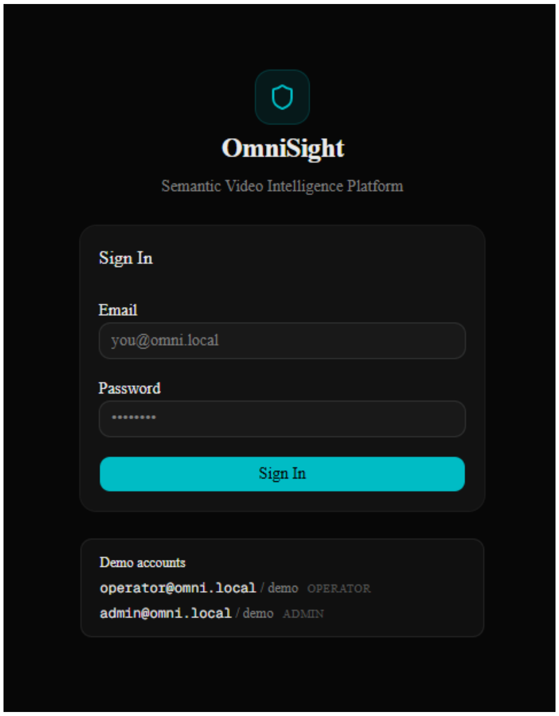
Term	Definition
AI City Challenge	A public traffic dataset of synchronized, multi-camera footage of road incidents; the only OmniSight dataset with overlapping camera views, used to test cross-camera forensic reconstruction.
Anomaly detection	Finding video segments that deviate from normal behavior, without relying on a complete list of every possible abnormal event.
ANN (Approximate Nearest Neighbor) search	A fast way to find the stored vectors most similar to a query vector, trading a little accuracy for large gains in speed.
CLIP	An early vision–language model that learned to match images and text by training on about 400 million image–text pairs from the web.
CNN (Convolutional Neural Network)	A neural network that detects patterns by sliding small filters over an image.
Cross-domain transfer	Reusing knowledge a model learned in some settings (for example, surveillance and sports) to perform in a new, unseen setting (for example, traffic).
Curriculum learning	Training a model on easy examples first and progressively harder ones, so it learns more reliably.
Docker Compose	A tool that defines and runs an application made of several containers from a single configuration file.
DoTA	A traffic dataset of ego-centric (dash-cam) driving videos with labeled anomalies and exact start and end times.
Dual encoder	A pair of networks — one for images, one for text — trained so that matching image–text pairs produce similar outputs.
Embedding	A list of numbers (a vector) that represents the meaning of an image, clip, or text, so that similar items sit close together.
Event anticipation	Predicting what is about to happen from a partially observed video, before the event completes.
FastAPI / FastMCP	Python frameworks used to expose OmniSight's features as a web API (FastAPI) and as MCP tools for external agents (FastMCP).
Forensic dossier	A structured, evidence-grounded incident report that assembles footage from one or more cameras into a cited timeline.
Grounding	Tying a model's output to specific retrieved evidence, so each statement is traceable and less likely to be invented.
Hallucination	A fluent but unsupported statement produced by a language model that is not backed by real evidence.

HNSW (Hierarchical Navigable Small World)	A layered graph index that finds approximate nearest neighbors quickly, even across billions of vectors.
InternVideo2	A video foundation model that encodes a short clip into a 768-dimensional vector capturing motion and temporal context.
ISO 25010	An international standard that organizes software quality into categories such as performance, reliability, security, and maintainability.
L2 normalization	Rescaling a vector to unit length so vectors can be compared fairly by direction rather than by magnitude.
MCP (Model Context Protocol)	An open standard that lets one AI system call another system's functions directly, without custom integration code.
Microservices	An architecture that splits a system into small, independent services that can be scaled or restarted on their own.
Open-vocabulary search	Searching video with any words the user types, rather than a fixed list of predefined labels.
Optical flow	A per-pixel measure of how things move between consecutive frames.
PFM (Predictive Forecasting Module)	OmniSight's forecasting component: a shared temporal model with per-domain heads that predicts event type, time-to-event, and confidence.
PostgreSQL	A relational database; in OmniSight it stores metadata and alert logs.
Qdrant	A vector database that stores embeddings and supports fast similarity search.
Qwen2.5-VL	A generative vision–language model that takes images or clips with a text prompt and produces grounded text; used in OmniSight's verification and reasoning.
RabbitMQ	A message broker that decouples video ingestion from AI processing by passing work through queues.
RAG (Retrieval-Augmented Generation)	Generating text that is first grounded in evidence retrieved from an external store, reducing invented content.
recall@5	The share of queries for which the correct result appears among the top five returned.
Reciprocal Rank Fusion (RRF)	A method that merges two ranked result lists by how highly each item ranks in each, rather than by raw scores that are not on comparable scales.
ROI (Region of Interest)	The part of a frame that matters; OmniSight uses motion to decide which frames and regions are worth processing.
Self-attention	A mechanism that lets each element in a sequence weigh how strongly it depends on every other element.
Self-supervised learning	Learning from raw, unlabeled data by predicting hidden parts from visible ones.
SigLIP 2	A vision–language model that encodes a single frame into a 1152-dimensional vector capturing scene content; the basis of OmniSight's semantic search.
SoccerNet	A sports dataset of broadcast soccer matches with densely timestamped events such as goals, fouls, and cards.

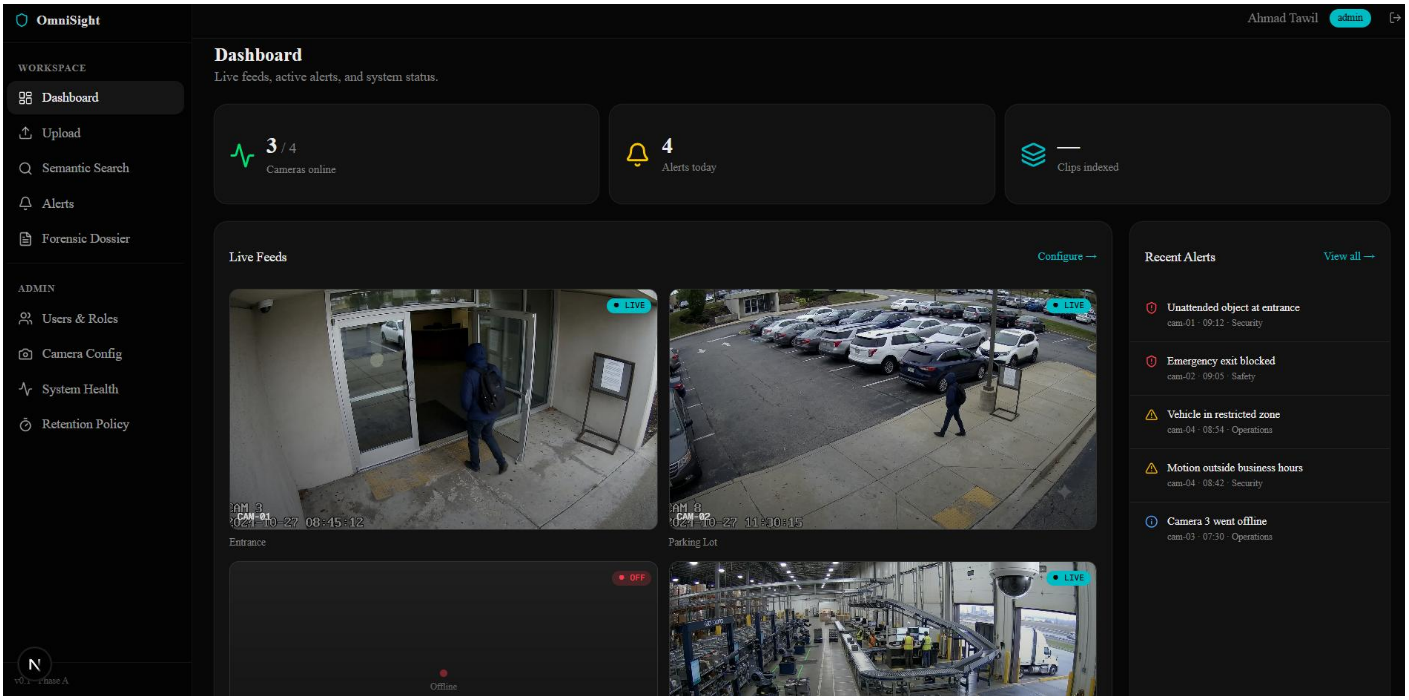
Temporal backbone	The shared part of the PFM that models how clip representations change over time, before per-domain heads make their predictions.
Time-to-event	The model's estimate of how long until a predicted event occurs, measured forward from the present moment.
Transformer	A neural network design in which every element of a sequence can attend to every other element through self-attention.
UCF-Crime	A surveillance dataset of about 1,900 untrimmed CCTV videos spanning 13 anomaly classes.
Verification cascade	A sequence of increasingly expensive checks that filters predicted events — from a cheap similarity test up to human review — removing false alarms early.
VIRAT	A surveillance dataset of stationary outdoor camera footage with object tracks and event labels; OmniSight's primary dataset for semantic-search evaluation.
VLM (Vision–Language Model)	A model that understands images or video together with text.
Zero-shot	A model's ability to handle categories or queries it never saw during training.

Appendix D - UI / Dashboard mockups

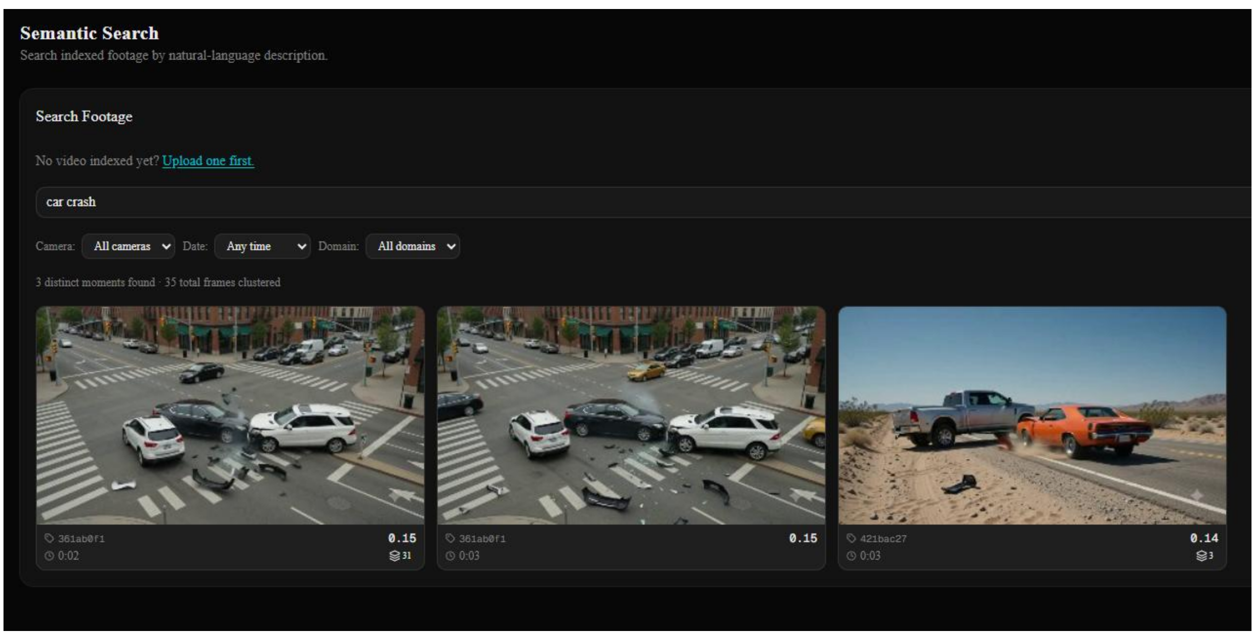
Login: The entry screen; every user and external system authenticates here before reaching any feature, and the account's role (operator or administrator) determines what they can access.

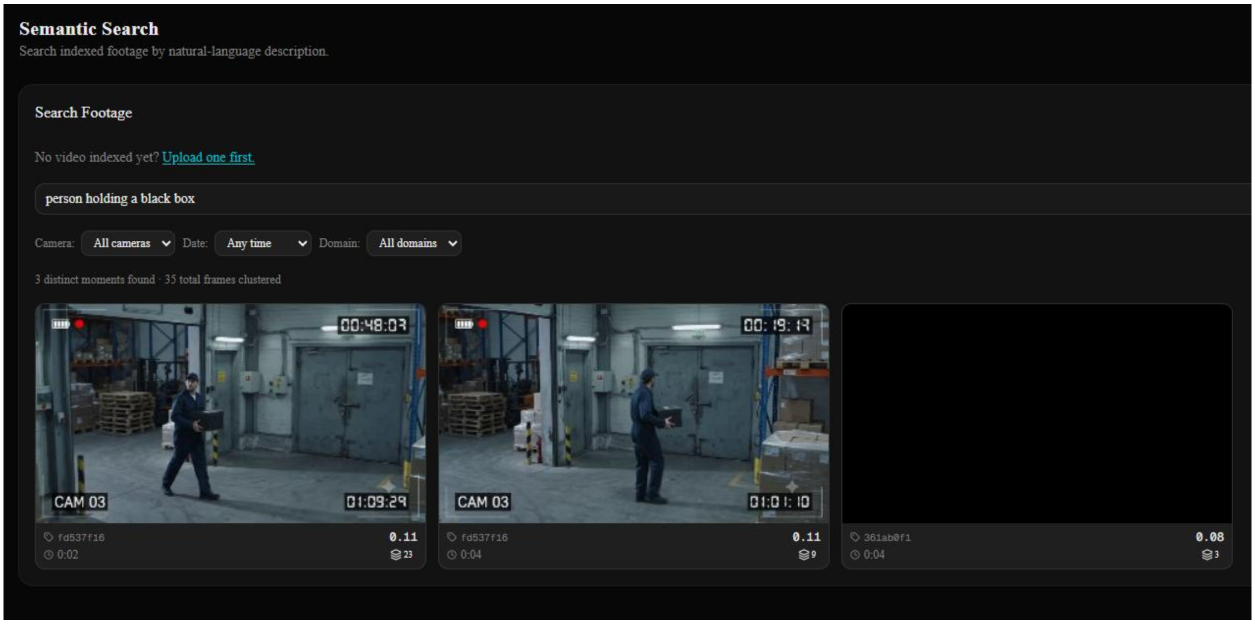


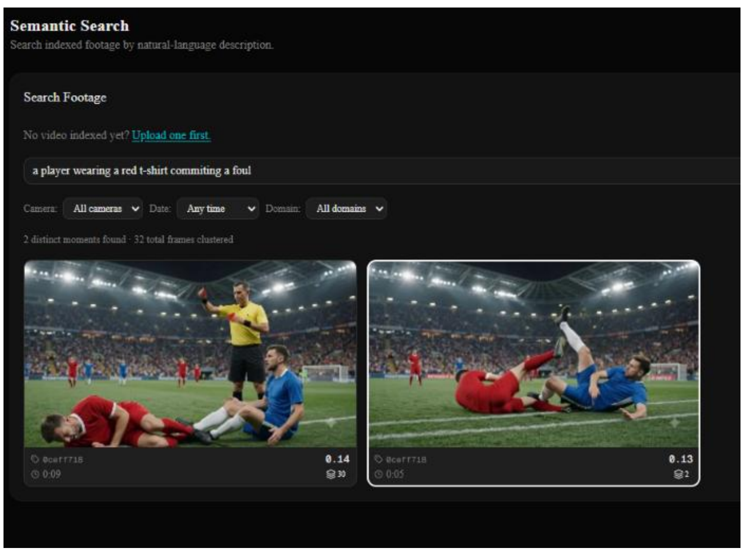
Dashboard: The operator's landing view, showing live camera feeds, headline metrics (active cameras, alerts, indexed clips, average search time), and a panel of the most recent alerts.

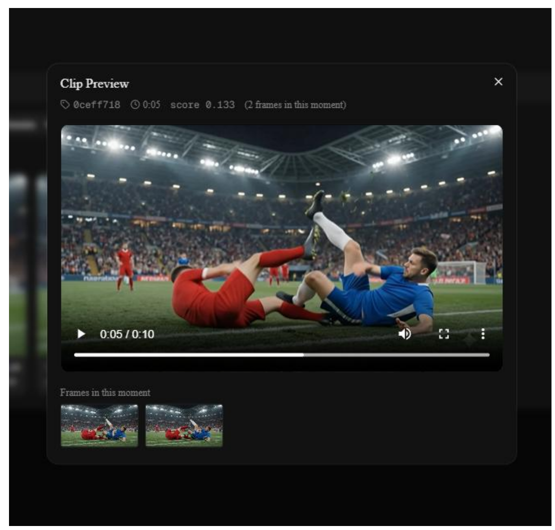


Semantic Search: Users find moments by typing a plain-language query; results are ranked clips with their camera, timestamp, and match score, retrieved using the SigLIP 2 frame embeddings.

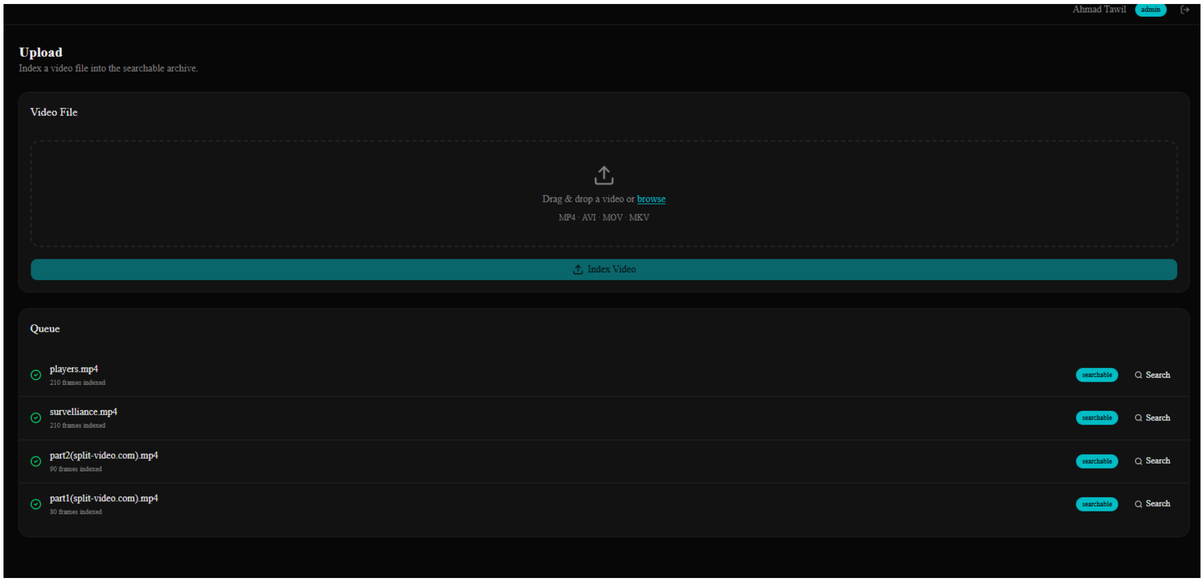



Semantic Search
Search indexed footage by natural-language description.
Search Footage
No video indexed yet? Upload one first.
person holding a black box
Camera: All cameras
Date: Any time
Domain: All domains
3 distinct moments found · 35 total frames clustered
00:48:03
CAM 03
01:09:29
CAM 03
fd537f16
0:02
0.11
23
fd537f16
0:04
0.11
9
361ab0f1
0:04
0.08
3

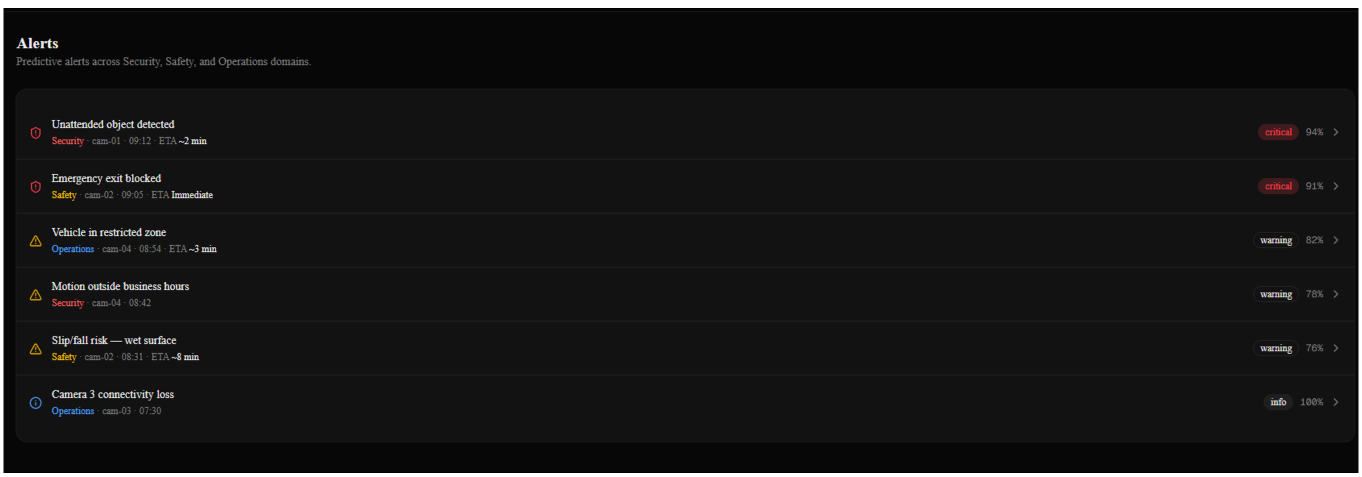

Semantic Search
Search indexed footage by natural-language description.
Search Footage
No video indexed yet? Upload one first.
a player wearing a red t-shirt commiting a foul
Camera: All cameras
Date: Any time
Domain: All domains
2 distinct moments found · 32 total frames clustered
0.14
0:09
30
0.13
0:05
2


Clip Preview
0ceff718
0:05
score 0.133
(2 frames in this moment)
0:05 / 0:10
Frames in this moment

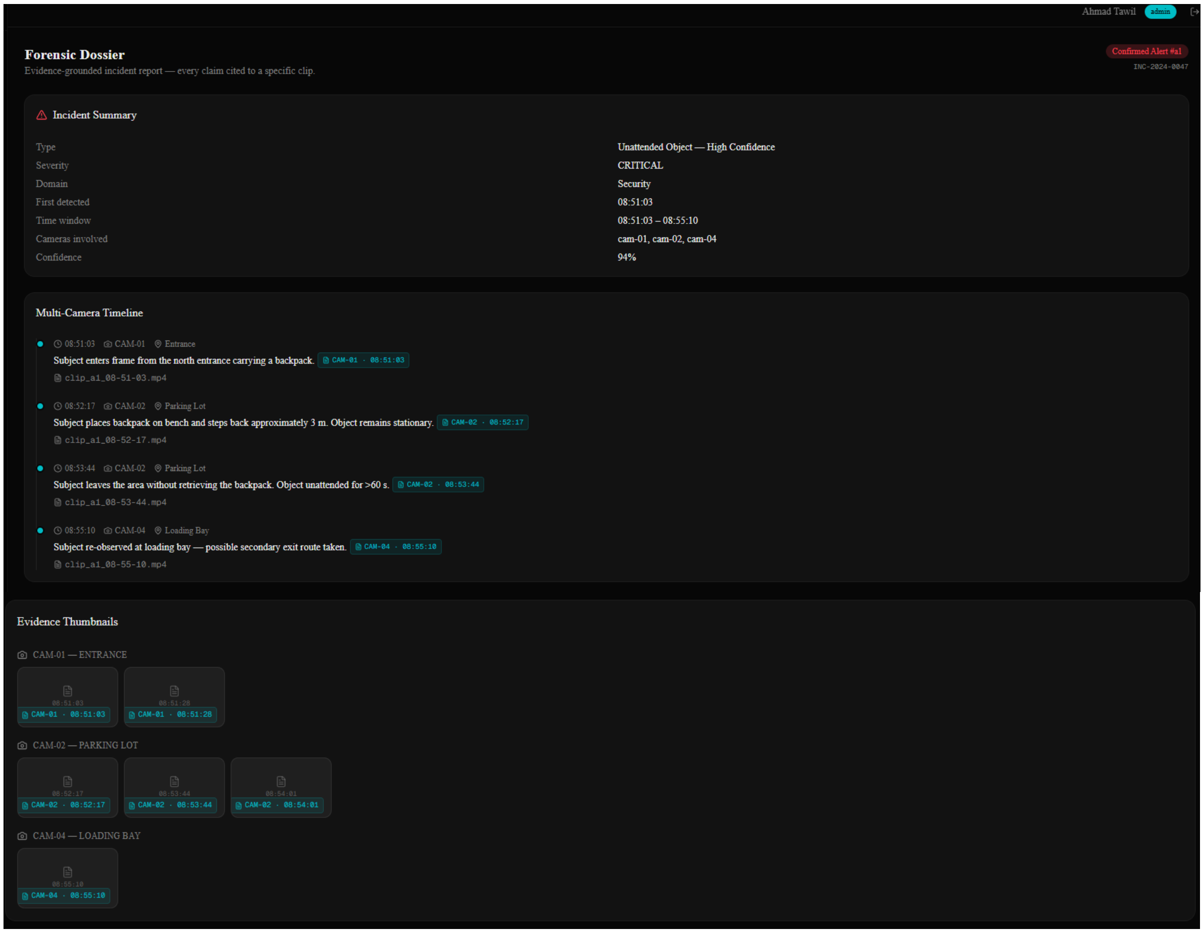
Upload: Operators add video by drag-and-drop or by connecting a live stream, and a processing queue shows each file moving from upload through indexing to searchable.



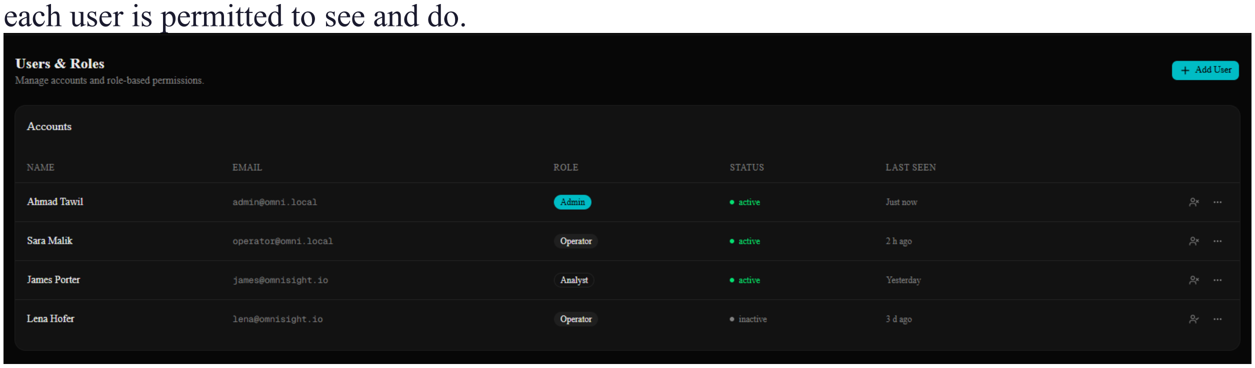
Alerts: Predictive alerts are listed with their event type, time-to-event, and confidence; selecting one opens its detail, including the triggering clip, the verification reasoning, and the operator's confirm or dismiss action.



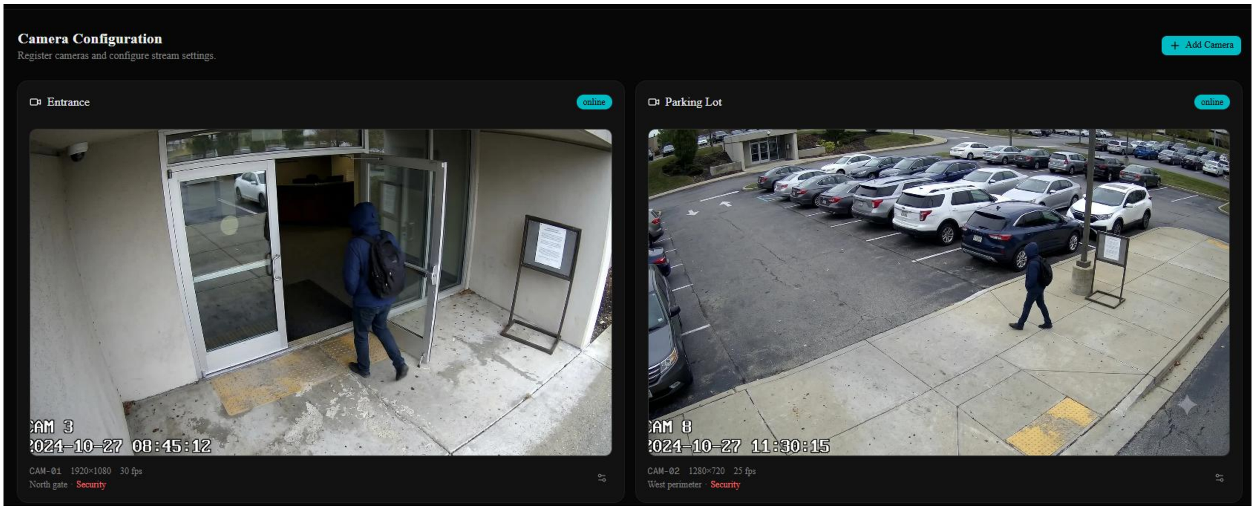
Forensic Dossier: After an alert is confirmed, the system assembles a cited, multi-camera incident report in which every claim links back to the specific clip or frame it was drawn from.



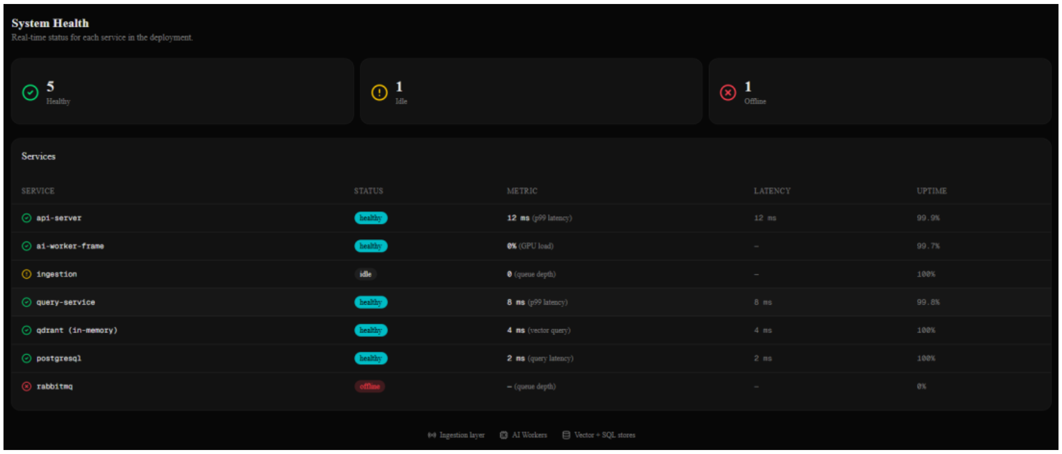
Users & Roles (Admin): Administrators create accounts and assign roles, which control what each user is permitted to see and do.



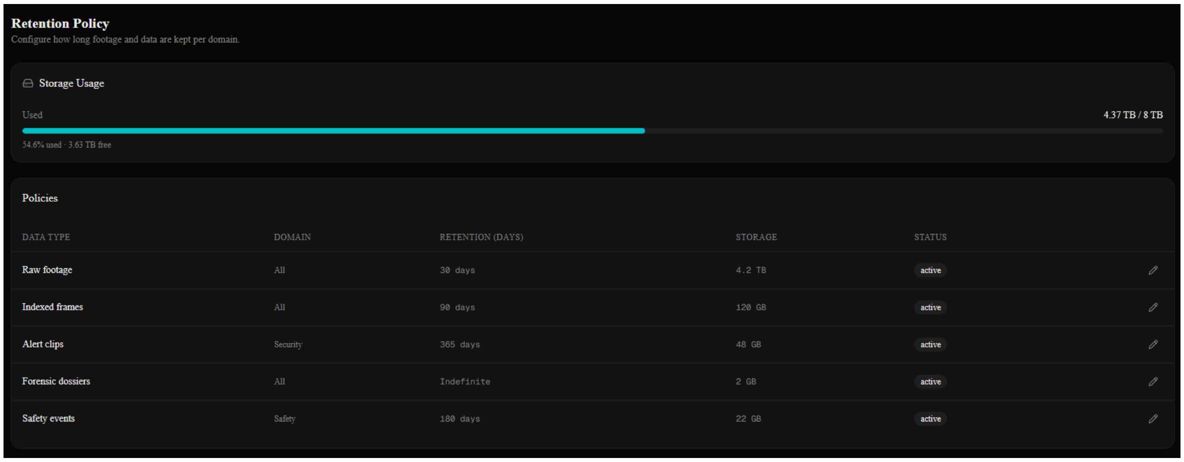
Cameras (Admin): Administrators register cameras and configure their location, stream source, and domain, and can see which are online.



System Health (Admin): Administrators monitor the operating status of each service in the platform, mirroring the deployment architecture described in §3.3.2.



Retention (Admin): Administrators set how long indexed footage is kept per domain before it is automatically purged.



Appendix E - Used AI Tools and Prompts

AI assistants were used during Phase A as auxiliary tooling — Perplexity to build the literature foundation, then Claude to turn that foundation into a connected engineering design — under continuous human review. All scientific decisions and final wording remain the team's authorship.

E.1 Perplexity

Used first, to build the literature base of the book: finding the foundational and recent papers behind every project component and gathering trusted references.
Example prompts:

- "Find the main papers behind vision-language models for open-vocabulary video search."
- "Recent work on event anticipation and predictive forecasting in video."
- "Datasets for traffic anomaly detection with precursor annotations."

E.2 Claude (Anthropic)

Used after the literature, for the engineering phase: taking the project idea and making sure every piece fits — algorithms connecting into one pipeline, datasets matching the algorithms that consume them, and diagrams reflecting the actual design.
Example prompts:

- "Here are my six algorithms — make sure they connect into one pipeline and the inputs/outputs line up."
- "I am using VIRAT, UCF-Crime, SoccerNet, DoTA, and AI City Challenge. Does each one actually match the algorithm and success metric I assigned it to?"
- "Give me mermaid code for the deployment architecture: frontend, backend with 4 services, data tier, RabbitMQ between them."

E.3 Workflow

Perplexity came first — to discover and verify external sources that grounded the literature review and the design choices. Claude came second — to take those grounded ideas and make sure the system fits together end-to-end. Every value taken from a model response was cross-checked against the original paper before being entered into the book.